Input Method Configuration Overview

This document presents the Input Method Editor (IME) system used in Scaleform 4.0, describing how Scaleform IME can be integrated into end user application and customized to match application UI style.

Author: Ankur Mohan, Michael Antonov
Version: 4.01
Last Edited: June 3, 2011

Copyright Notice

Autodesk® Scaleform® 4

© 2011 Autodesk, Inc. All rights reserved. Except as otherwise permitted by Autodesk, Inc., this publication, or parts thereof, may not be reproduced in any form, by any method, for any purpose.

Certain materials included in this publication are reprinted with the permission of the copyright holder.

The following are registered trademarks or trademarks of Autodesk, Inc., and/or its subsidiaries and/or affiliates in the USA and/or other countries: 3DEC (design/logo), 3December, 3December.com, 3ds Max, Algor, Alias, Alias (swirl design/logo), AliasStudio, Alias|Wavefront (design/logo), ATC, AUGI, AutoCAD, AutoCAD Learning Assistance, AutoCAD LT, AutoCAD Simulator, AutoCAD SQL Extension, AutoCAD SQL Interface, Autodesk, Autodesk Intent, Autodesk Inventor, Autodesk MapGuide, Autodesk Streamline, AutoLISP, AutoSnap, AutoSketch, AutoTrack, Backburner, Backdraft, Beast, Built with ObjectARX (logo), Burn, Buzzsaw, CAiCE, Civil 3D, Cleaner, Cleaner Central, ClearScale, Colour Warper, Combustion, Communication Specification, Constructware, Content Explorer, Dancing Baby (image), DesignCenter, Design Doctor, Designer's Toolkit, DesignKids, DesignProf, DesignServer, DesignStudio, Design Web Format, Discreet, DWF, DWG, DWG (logo), DWG Extreme, DWG TrueConvert, DWG TrueView, DXF, Ecotect, Exposure, Extending the Design Team, Face Robot, FBX, Fempro, Fire, Flame, Flare, Flint, FMDesktop, Freewheel, GDX Driver, Green Building Studio, Heads-up Design, Heidi, HumanIK, IDEA Server, i-drop, Illuminate Labs AB (design/logo), ImageModeler, iMOUT, Incinerator, Inferno, Inventor, Inventor LT, Kynapse, Kynogon, LandXplorer, LiquidLight, LiquidLight (design/logo), Lustre, MatchMover, Maya, Mechanical Desktop, Moldflow, Moldflow Plastics Advisers, MPI, Moldflow Plastics Insight, Moldflow Plastics Xpert, Moondust, MotionBuilder, Movimento, MPA, MPA (design/logo), MPX, MPX (design/logo), Mudbox, Multi-Master Editing, Navisworks, ObjectARX, ObjectDBX, Opticore, Pipeplus, PolarSnap, PortfolioWall, Powered with Autodesk Technology, Productstream, ProMaterials, RasterDWG, RealDWG, Real-time Roto, Recognize, Render Queue, Retimer, Reveal, Revit, RiverCAD, Robot, Scaleform, Showcase, Show Me, ShowMotion, SketchBook, Smoke, Softimage, Softimage|XSI (design/logo), Sparks, SteeringWheels, Stitcher, Stone, StormNET, StudioTools, ToolClip, Topobase, Toxik, TrustedDWG, U-Vis, ViewCube, Visual, Visual LISP, Volo, Vtour, WaterNetworks, Wire, Wiretap, WiretapCentral, XSI.

All other brand names, product names or trademarks belong to their respective holders.

Disclaimer

THIS PUBLICATION AND THE INFORMATION CONTAINED HEREIN IS MADE AVAILABLE BY AUTODESK, INC. “AS IS.” AUTODESK, INC. DISCLAIMS ALL WARRANTIES, EITHER EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO ANY IMPLIED WARRANTIES OF MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE REGARDING THESE MATERIALS.

How to Contact Autodesk Scaleform:

Document	Input Method Configuration Overview
Address	Scaleform Corporation 6305 Ivy Lane, Suite 310 Greenbelt, MD 20770, USA
Website	www.scaleform.com
Email	info@scaleform.com
Direct	(301) 446-3200
Fax	(301) 446-3199

Table of Contents

1 Introduction 1

2 Scaleform IME Features 2

3 Scaleform 4.0 Changes 2

4 IME Setup 3

4.1 IME Support under Windows 3

4.2 Hiding Default IME Windows 4

4.3 IME Sample Application 5

4.4 Scaleform IME Files and Dependencies 6

4.4.1 Building Win32 IME Source 7

5 Using Input Method Editors 9

5.1 IME UIs 9

5.2 4.2. The Composition String 10

5.3 The Reading Window 11

5.4 The Candidate List 12

6 Integrating IME into your Game 13

6.1 IME Interaction Dataflow 13

6.2 Win32 IME Code Integration 14

6.3 Handling Focus Changes 17

6.3.1 Focus Changes within a Movie 17

6.3.2 Focus Changes across Movies 17

7 Customizing the IME User Interface 18

7.1 Composition String Styles 18

7.2 Candidate List Customization 19

7.3 Disabling IME on specific TextFields 21

8 IME.swf UI File Structure and Customization 22

8.1 Candidate List Implementation 23

8.2 IMECommands 25

8.3 Font Management 25

9 Data Driven IME Registration System 26

9.1 Existing System 26
9.2 Data Driven IME Support 27
9.3 Implementation 27
10 Scaleform Support for Flash IME Class 32
10.1 IME support on Flash 32
10.2 Properties 33
10.3 General Observations 33
10.3.1 get/setEnabled and get/setConversionMode 33
10.3.2 OnIMEComposition Listener 35
10.3.3 OnSwitchLanguage Listener 36
10.4 Using disableIME vs. setEnabled() 36
11 Function Reference 37
11.1 setIMECompositionStringStyle () method 37
11.2 setIMECandidateListStyle () method 38
11.3 getIMECompositionStringStyle () method 39
11.4 getIMECandidateListStyle () method 40
12 Appendix A: Language Bar and Status Window in Scaleform 41
12.1 Using the System Language Bar and Status Window 42
12.2 Language Bar Demo Shipped With Scaleform 3.x+ 45
12.3 Implementation of the Language Bar and Status Window in Scaleform 46
10.3.1 Using the Language Bar in your Game 54
13 Appendix B: IME Screenshots 55
13.1 Chinese (Simplified) Vista 55
13.1.1 Simplified Chinese IMEs 55
13.1.2 Traditional Chinese IMEs 55
13.2 Chinese (Traditional) Vista 56
13.2.1 Simplified Chinese IMEs 56
13.2.2 Traditional Chinese IMEs 57
13.3 Chinese (Simplified) XP 57
13.3.1 Simplified Chinese IMEs 57
13.3.2 Traditional Chinese IMEs 58
13.4 Chinese (Traditional) XP 58
13.4.1 Simplified Chinese IMEs 58

13.4.2 Traditional Chinese IMEs 59
13.5 Korean IMEs 59
13.5.1 Korean Vista 59
13.5.2 Korean XP 59
13.6 Japanese IMEs 60
13.6.1 Japanese XP 60
13.6.2 Japanese Vista 61
14 Appendix C: Windows IME Setup 62
14.1 Windows XP 62
14.2 Windows Vista 64
15 Appendix D: IME Messages on Windows 65

1 Introduction

This document describes Input Method Editor (IME) implementation in Autodesk Scaleform. Input method editors are programs that allow users to input text in East Asian languages using a standard keyboard. Many of these languages, such as Chinese, Japanese, and Korean have large character sets that cannot be easily mapped to keyboard keys. To address this limitation, input method editors have been developed that assist text entry by displaying suggestions and modifying the input string as it is being typed. By using IME, uses can type ideographic characters in a word processor and chat with their friends in a massive multiplayer game.

The sections of this document are structured to follow the typical steps a user will take when working with IME in Scaleform. The main ideas covered in each section are outlined below.

- *Section 4: IME Setup* – explains the details of Scaleform IME SDK installation and guides the user through the sample application.
- *Section 5: Using Input Method Editors* – introduces the fundamental IME user interface elements such as the composition string, the reading window and the candidate list.
- *Section 6: Integrating IME into your Game* – discusses event handling setup required for IME and describes the code changes you will need to make to integrate it into your game.
- *Section 7: Customizing the IME User Interface* – describes the ActionScript extensions available in Scaleform for customizing the color and style of the IME popup windows.
- *Section 8: IME.swf UI file Structure and Customization* – describes the structure and implementation of IME.swf, a file that is responsible for displaying the popup windows used by IME.
- *Section 9: Data Driven IME Registration System* – describes the data driven system for registering IMEs supported by users.
- *Section 10: Scaleform Support for Flash IME Class* – describes the implementation of Flash IME class on Scaleform and the language and operating system specific differences between the outputs of the methods.
- *Section 11: Function Reference* – provides a detailed reference for ActionScript APIs used for customizing IME.

If you are an user who is intimately familiar with IME and would like to get started integrating it into your game, you should first skim over Section 2 to understand the SDK installation and then proceed directly to Section 5, which addresses the details of game integration. If you are less familiar with IME and would like to get a solid understanding of the fundamental concepts involved, you should spend

more time reading Section 4 and playing with the sample application. Overall, the document is best understood when read from the beginning to the end.

2 Scaleform IME Features

Scaleform IME includes these IME features- the Language Bar that allows the user to change the current input language and IME and the Status Window for changing the conversion mode of the currently active IME. The addition of these features provides a complete IME solution that closely replicates the behavior on Windows and allows the game player to easily switch input languages and type in Asian characters in full screen mode. See Appendix A for detailed description on the new IME features.

3 Scaleform 4.0 Changes

The overall structure of IME implementation is relatively unchanged from 3.x. As explained in later sections, the user still

1. Creates an instance of the IMEManager.
2. Taps into IME related windows messages, creates Events and passes them to Scaleform.

Since Scaleform 4.0 supports both AS2 and AS3, we have provided the AS2 and AS3 versions of the *IMESample.swf* file (located in Bin/Data/AS2 or AS3). The structure of the flash files has been simplified. Since AS3 doesn't support scripts on MovieClip instances, these scripts have been moved to either AS files or frame scripts (for example see ListRow.as). Another major departure from 3.x is the removal of the specialized fscommand called "imecommand" that was used to send information from AS to C++. Instead, the function injection mechanism introduced in the Direct Access API is used. Using this mechanism, we inject C++ implementations for certain predefined ActionScript functions (for example, "SendIMEMessage" used in FxCandidateListBox.as) which are used as replacements for imecommand. From the user's perspective, it's important to understand that the user doesn't have to define or implement these functions in ActionScript, they are injected automatically by the core.

GFxIME supports customizing the appearance of the candidate list through ActionScript to match the current theme. In AS3, this customization is performed using the getIMECandidateListStyle and setIMECandidateListStyle functions which are static members of the custom IMEEx class (look at IMEEx.as in *AS3/IME/Scaleform/gfx*).

Also, please note that the flash IME class and the language bar is currently not implemented in AS3. These components are under development and we should be able to release both of them in the next release of Scaleform 4.0.

4 IME Setup

Chinese and Japanese IME is distributed as a separate add-on package for Scaleform, available from https://developer.scaleform.com/gfx?action=dl. IME distributions are named based on the Scaleform SDK version and the target platform they are compatible with, as in "gfx_4.0.xx_src_ime_msvc80.exe". To use IME, you need to download and install it on top of the regular Scaleform SDK.

Since IME is provided separately, existing licensed customers will need to contact support to get an IME evaluation key. Evaluation customers should not need to do this, since the same key will work for both packages.

Developers who are targeting Korean language under Windows should not need to install this package, since Korean input support is built into the core of Scaleform. However, supporting Korean still requires developers to follow the message passing procedures described in section 6.2, required for Korean input to work properly.

After installing the IME SDK you can rebuild the Scaleform solution to make all players IME aware. The distribution makes this possible by copying the IME libraries and headers to the corresponding Scaleform directories and defining the SF_ENABLE_IME macro in GFxConfigAddons.h. This is all that is necessary for Scaleform Player source to compile, since it already includes the logic required to enable IME. For more information on files and libraries provided as a part of the IME distribution refer to Section 4.4, Scaleform IME Files and Dependencies.

4.1 IME Support under Windows

Scaleform IME implementation relies on operating system APIs for managing the composition string and providing candidate list choices. Under Windows, IME support will not be installed by default unless you are using an Asian version of Windows. If that is the case, you should install IME support before using the Scaleform IME SDK. Instructions on how to install IME are provided in Appendix C. These might also be useful if you already have some IMEs installed and wish to install additional IMEs for testing purposes.

In Windows, multiple IME implementations can be installed at the same time. The Scaleform IME SDK has been tested with the following standard IMEs and third party IMEs on Windows XP and Vista:

Standard Windows IMEs

- Japanese IME (MSIME 2002, 2003, 2007)
- Chinese PRC Microsoft Pinyin 3.0, 2007

- Chinese Simplified ABC
- Chinese Simplified NeiMa
- Chinese Simplified QuanPin
- Chinese Simplified ShuangPin
- Chinese Traditional ChangJie
- Chinese Traditional Phonetic
- Chinese Traditional New ChangJie
- Chinese Traditional New Phonetic
- Chinese Traditional DaYi
- Chinese Traditional Quick
- Chinese Traditional Array

Third Party IMEs

- ATOK Japanese IME
- Google Pinyin IME
- Sogou Pinyin IME

Scaleform IME has been tested to fully support the extra keys on the Japanese 106 keyboard as well as the “Direct Kana” input mode.

A list of screenshots of the supported IME's on the various test platforms are presented in Appendix B of this document.

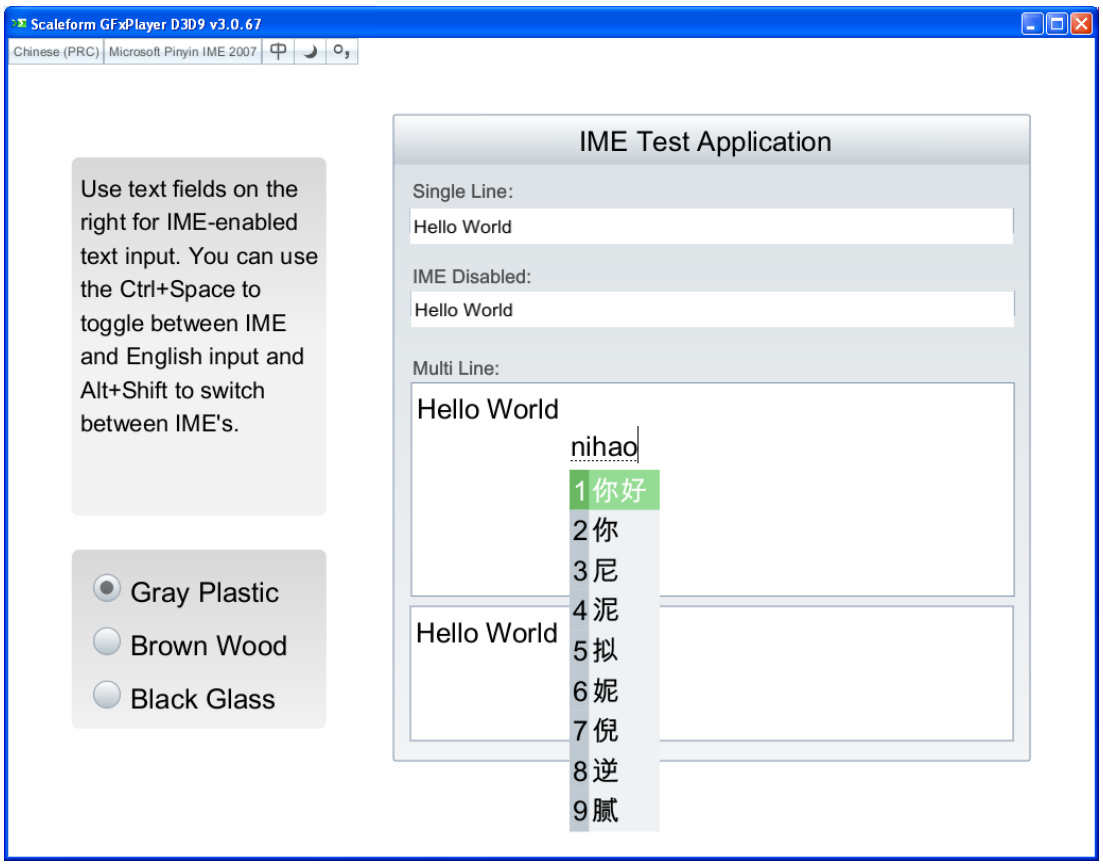
4.2 Hiding Default IME Windows

An application that intercepts IME messages and draws its own IME UI windows is called “IME Aware”. Since an application that uses GFxIME is IME aware, it does not need the system IME windows anymore. More ever, as these system IME windows are drawn using Windows GDI, they cause annoying flickering when they try to appear on top of the DirectX window used by the Gaming application. The documentation on the Input Method Manager (IMM) API on MSDN suggests using the WM_IME_SETCONTEXT message and setting various bits of the lParam parameter to hide the IME

windows. In practice, this approach is ineffective and fails to hide the system IME windows. After trying a variety of other approaches, we've found that passing the WM_IME_SETCONTEXT message to the default window procedure hides the UI windows for all the IME's without affecting the performance of the IME.

4.3 IME Sample Application

Before performing the SDK integration, it is instructive to run the sample player and experiment with IME behavior. A pre-built IME enabled player is shipped with the installation and is accessible as "IME Demo" from the Windows start menu; when launched it will automatically load the IME Sample file. Alternatively, users can also load the sample by rebuilding the Scaleform Player SDK after IME installation and dragging the IMESample.swf into it from the Bin\Data\{AS2or AS3}\IME folder. Once the file is loaded, you should see the following picture.



The purpose of this IMESample file is to provide text fields for IME testing and to illustrate customization of the IME pop ups according to the chosen window theme. While running the sample, you can use the system language bar to switch to the input language of your choice to test IME. Once that is done, you can enter text in the "Single Line" or "Multi Line" text fields. Note that IME is disabled in the second single line text field, so you can only type English characters there regardless of the selected input language.

In more detail, the IME Sample application demonstrates the following:

- Behavior of the IME reading and candidate list UI widgets that popup as you enter Asian text.
- Customization of UI Widgets to match window theme. Users can change the display theme by clicking on the theme selection radio buttons on the left and observe the colors of the candidate list and the reading window change to compliment the chosen theme.
- Disabling IME for specific text fields. In this sample, IME has been disabled on the second single line text field.
- Customization of the composition string. The composition string in the first single line text field will appear in red with green underline, while the clause text will appear in light blue color with dithered thick underline. This is different from the default behavior.
- The IME UI widgets obtain their resources from the underlying text field and their appearance can be customized to match that of the text field. The single line text fields use a smaller font than the multi line fields and the font used in the candidate list and reading window matches that of the text field.

For details on customization of IME popup windows, refer to Section 7: Customizing the IME User Interface.

4.4 Scaleform IME Files and Dependencies

During installation, Scaleform IME SDK copies a number of library, header and resource files to the Scaleform SDK subdirectory. Since you will need to use these files for linking and integration, this section explains the various IME related resources that come with the installation and how to use them. To summarize, the IME distribution installs the following resources:

- libgfx_ime.lib (Lib\Win32\)
- Public headers needed to use IME (Include\)
- Flash assets, including ime.swf and demo files (Bin\Data\{AS2 or AS3}\IME\)

In addition, customers with IME source license also get access to the source code needed to build the libgfx_ime.lib library. The source files are located in the Src\GFxIME\ subdirectory.

The following header and library files are needed to use Scaleform IME:

Header Files Needed

- GFx.h (Include\)
- GFx_IMEManagerWin32.h (Src\GFxIME\ )

Library Files Needed

- libgfx_ime.lib
- imm32.lib (Microsoft Visual Studio 8\VC\PlatformSDK\Lib\)
- uuid.lib (Microsoft Visual Studio 8\VC\lib\)

Asset Files Needed

- ime.swf or ime.gfx (Bin\ Data\{AS2 or AS3}\IME\)

The main header file you will include for IME on Windows is GFx_IMEManagerWin32.h. This file defines the GFxIMEManagerWin32 class responsible for implementation of the abstract GFx::ASIMEManager on the Windows. Complete details on how to use these classes are provided in Section 6, Integrating IME in your Game.

libgfx_ime.lib is the main library provided by the Scaleform IME SDK; you will need to link this library to build an executable with IME. The imm32.lib and uuid.lib are additional required libraries distributed with Microsoft Visual Studio. Since IME implementation on Windows XP uses the Input Method Manager (IMM) APIs, you will need to link to IMM functions defined in imm32.lib. Similarly, IME implementation on Windows Vista uses the Text Services Framework (TSF), requiring definitions contained in uuid.lib.

The IME implementation relies on the ime.swf asset file for displaying the reading window and candidate list popups; you will have to distribute a version of this file with your application. The details on the structure of this file are provided in Section 8, IME.swf UI File Structure.

4.4.1 Building Win32 IME Source

Customers who have access to IME source code will get the source files needed to build the GFxIME library file. Our SDK solution includes GFxIME project to build libgfx_ime.lib and can be used once IME source code is installed. To build the libgfx_ime.lib from source, developers will need to install the Microsoft Platform SDK.

The Platform SDK is required because IME implementation for Windows Vista uses some features of the Text Services Framework (TSF). TSF is a COM based system that needs the msctf.h header file distributed as part of Microsoft Platform SDK. The Platform SDK can be downloaded from:

www.microsoft.com/downloads

Click the “Developer Tools” link under the Browse for Downloads section. In the drop down list under “Show downloads for”, select Microsoft Platform Software Development Kit. Select “Microsoft Windows SDK Update for Windows Vista” for Windows Vista and “Windows® Server 2003 R2 Platform SDK Full Download” for Windows XP.

5 Using Input Method Editors

This section presents a high level overview of IME, describing the Input Method Editor behavior from the end user perspective and defining IME terminology. Reading this section is critical for non-Asian developers who haven't had prior experience using IME and are tasked with integrating it into their Scaleform based application.

As was mentioned in the introduction, input method editors are programs that allow users to input text in East Asian languages, such as Chinese, Japanese, and Korean using a standard keyboard. Use of the IME technology relieves end users of the need to remember all possible ideographic character values. Instead, IME monitors user keystrokes as they type and anticipates the characters the user might want. When several similar choices are available, IME presents a list of candidate strings giving the user an opportunity to choose an appropriate one.

On desktop computers IME support is traditionally provided by the operating system, which monitors keystrokes and displays system popup windows with status information and candidate strings. When system display of popup windows is inappropriate, applications can tap into system IME notifications and use them to manage their own IME windows, providing a custom UI experience. On consoles, the operating system usually provides only the most rudimentary input method APIs, leaving it up to developers to display candidate lists when appropriate.

When IME popup support is provided by the OS, it is usually tied to system text field controls. Since Scaleform is a portable UI solution, Scaleform needs to implement its own IME functionality to both support text fields on all platforms and to provide precise control over pop-up window positioning. Moreover, rendering its own IME reading windows and candidate lists allows developers to customize their appearance to fit the game and helps avoid flickering in full screen mode.

5.1 IME UIs

The four main UI items that the user interacts with when using IME are the *composition string*, the *reading window*, the *candidate list* and the *status window*. The status window enables selection of the desired IME and is normally provided as a part of the operating system; under Windows, it corresponds to the desktop language bar. Before entering Asian text, users will often use keyboard shortcuts or the language bar to select the desired input language and IME conversion mode. With the target language selected, users will proceed to enter text into the UI text fields and interact with the reading window and candidate lists that popup.

Unlike the system status window, the reading window and candidate list pop-ups are rendered directly by Scaleform and are displayed as a part of or adjacent to the input text field. The behavior of these popups is illustrated in Figure 1 and explained in detail on the next page.

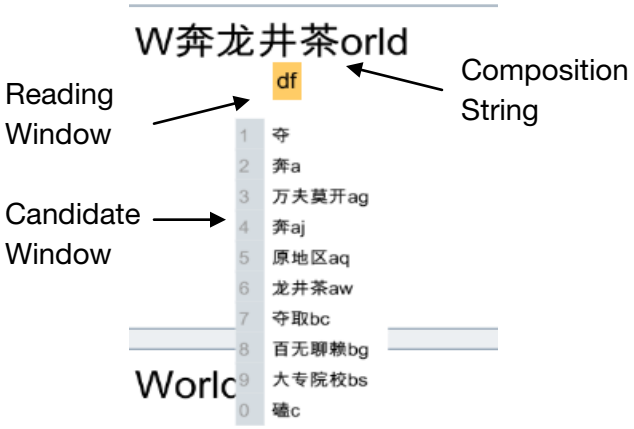


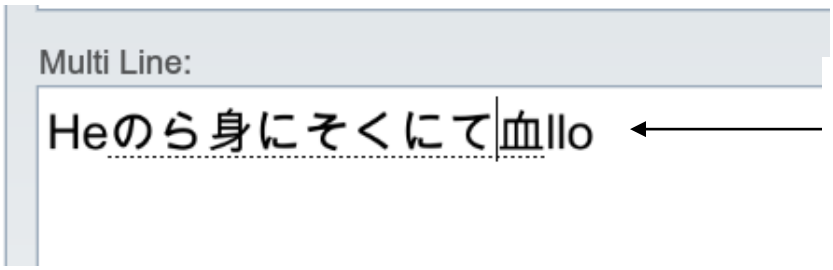
Figure 1: Candidate Window and Reading Windows

The different IME elements illustrated in Figure 1 have the following purpose:

- **Composition String** – contains the collection of characters that the user has composed with the IME. In Scaleform, the composition string appears as part of the current text field and is often underlined or highlighted based on user input, as explained in the next section.
- **Reading window** – contains keyboard keystrokes that change as each key is pressed; this string is converted into composition characters when the spacebar is pressed.
- **Candidate List Window** – displays a list of candidate characters that correspond to the current clause in the composition string. When the user selects the intended character from this list IME adds it to the composition string display.

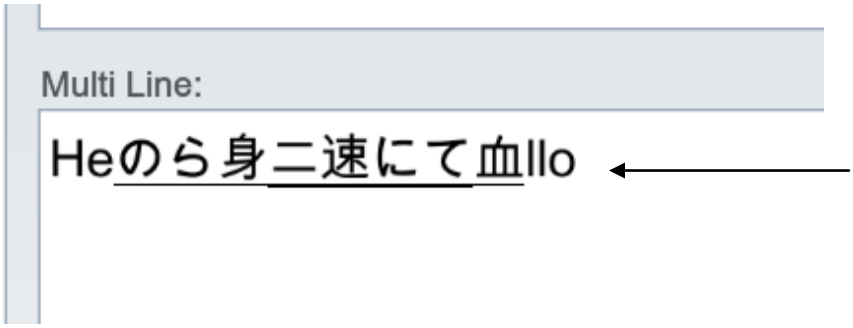
5.2 4.2. The Composition String

The composition string is a highlighted part of the current text field that the user interacts with while using IME; this string can be converted to final characters by pressing a dedicated key, such as a spacebar. The exact highlighting and input key behavior applied varies greatly with IME in use. The picture below illustrates the default composition string used with Japanese IME.



When text is entered with Japanese IME characters appear with a dotted underline.

In Japanese, the composition string can be broken down into one or more "clauses". A clause is the smallest combination of characters that the IME can convert to a final character. The currently active clause is highlighted with a special underline style and can be selected with arrow keys before the string is finalized. An example of clause selection is illustrated on the next page.



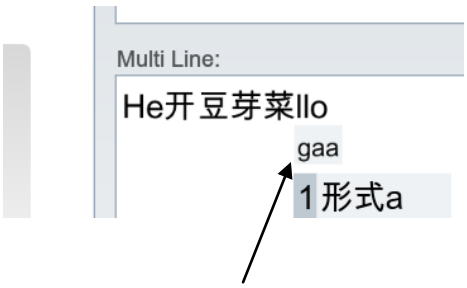
Upon conversion, composition string appears with a solid underline and the current clause with a bold underline.

Although the notion of the “clause” string is not used directly by most Chinese IMEs, alternative IME implementations often include similar substring selection behavior. While Scaleform tries to handle input consistently across all platforms, the exact details of IME handling can vary with the operating system and even application in use.

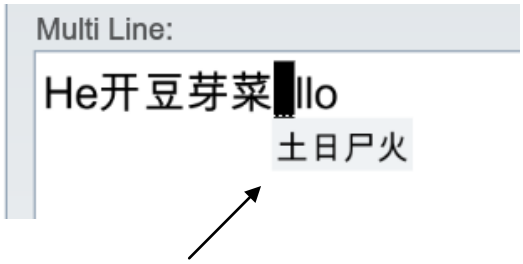
ActionScript developers should note that access of TextField properties will not consider the composition string until it is finalized. TextField.length and TextField.text properties, for example, will behave as if the composition string did not exist. Furthermore, unless the composition style is explicitly modified, the style of the character before the composition string will be used for the entire string. When the text field is empty, the default text format for the text field is used. The details of changing the composition string style are provided in Section 7.1,Composition String Styles.

5.3 The Reading Window

The reading window contains the key strokes from the keyboard. This is used in Chinese Simplified and Traditional IMEs. In Chinese Simplified IMEs the reading window contains the actual letters the user types on the keyboard, while in Chinese Traditional IMEs it contains the Unicode character corresponding to the keystroke. These two scenarios are illustrated below.



Chinese Simplified (Microsoft PinYin 3.0) Reading Window containing actual key-strokes.



Chinese Taiwan (New ChangJie) Reading Window contains Unicode Characters.

The user can convert reading window text into composition characters usually by pushing space or enter key.

5.4 The Candidate List

The candidate list window contains a list of “candidates”, or alternative character sequences, for the selected characters in the composition string. It is shown as a long vertical popup in a picture below.



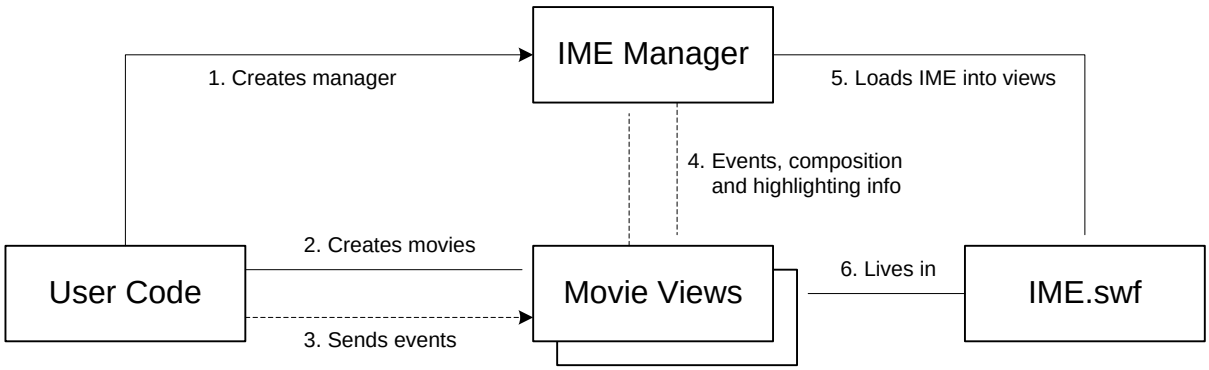
Depending on the IME selected, the candidate list will either pop up automatically of after a dedicated key is pressed. Once the list is displayed, the user can use the numeric keys or the mouse cursor to select the desired string; they can also use the PageUp and PageDown keys if the list is long.

6 Integrating IME into your Game

This section describes how to use IME in your game application. The section begins by introducing the IME Manager class responsible for IME implementation and explaining IME event data flow between the manager, movie views and the user code. Given the understanding of IME dataflow, subsection 6.2 describes the code changes necessary to integrate IME support into your code. The remaining subsections discuss the details of focus handling, which becomes particularly important with IME, and provide additional information about IME implementation.

6.1 IME Interaction Dataflow

The GFx::ASIMEManager is the IME implementation class used to manage the display of IME user interface and to abstract away system-specific details of IME implementation. The diagram below illustrates the interaction between the user code, IME manager, movie views and the IME UI windows.



In the diagram solid arrow describe the creation/ownership relationship, while the dashed arrows illustrate the flow of events and message data. We will now describe the interactions labeled by the numbered arrows in more detail.

1. During initialization, user code creates the IME manager and sets it on the loader. Here the user also specifies the path for the SWF/GFX file containing the IME UI implementation.

2. User creates one or more movie views and calls HandleEvent(SetFocus) on the currently focused view, associating it with the IME manager.

3. User notifies the movie view about input events including IME events and focus changes by calling HandleEvent. Although most events are passed in system independent format, IME events are system-specific and are passed only to the currently focused movie view.

4. Internally, IME manager communicates with the movie view, informing the focused text field about the state of the composition string, current clause, highlighting information and other

IME details. The movie view delegates system-specific IME Events handling to the IME manager, allowing those events to be processed correctly.

5. When the IME implementation needs to display the reading window or candidate list popups, it loads the IME UI file into the focused movie view. From that point on, it uses Invokes and the internal *imecommand* interface to communicate with the popup windows about events such as content changes or the user clicking on a row of the candidate list.

6. The IME UI windows live within the active movie view until they are closed by IME Manager or the focus changes. Living within a particular movie view enables popups to be rendered as a part of the scene and be customized to the appearance of its user interface.

Although it is helpful to understand the flow of events between the IME components, most of the details described in steps four through six are handled automatically by the IME implementation. The different aspects of composition string handling, including obtaining candidate string data from the system, string highlighting, display of popups, management of the cursor position, and candidate finalization are performed by the C++ logic separated between both libgfx_ime.lib and Scaleform library core. Developers should have no need to modify source code of these libraries.

6.2 Win32 IME Code Integration

As can be seen from the dataflow diagram, the only thing users need to do support IME is initialize the IME manager and the pass relevant IME system events to the movie view. Under Windows, this means creating the GFxIMEManagerWin32 object and passing the various WM_IME messages from the programs message loop into GFx::Movie::HandleEvent. For a working example of how this is done, you can search for the use of the SF_ENABLE_IME macro in the FxPlayerAppBase.cpp source file.

In the earlier version of GFxIME shipped with Scaleform 2.1, user were required to create a Unicode application window for IME to work correctly. This can be done by using the Unicode version of the CreateWindow and RegisterWindow functions. This requirement has now been considerably simplified. With Scaleform 2.2 and above, users only need to use Unicode version of the PeekMessage (PeekMessageW) function. No other Unicode Win API functions are needed.

The following actions need to be performed in your game/application to make it IME aware.

1. Create IME Manager, inform it about the IME UI file and set it on the loader. This is done by the following sample code:

```
// Create IMEManager after the application window has been set up
```

```
// since IME manager needs valid window handle as parameter.
GFxIMEManager* pimemanager = new GFxIMEManagerWin32(app->hWND);

// For error logging. If error logging not desired, just pass null to
// Init. User must call Init though!
pimemanager->Init(Loader.GetLog());

if (pimemanager)
{
    pimemanager->SetIMEMoviePath("IME.swf");
    Loader.SetIMEManager(pimemanager);
    if (pMovie)
        pMovie->HandleEvent(GFx::Event::SetFocus);
}
```

Sending the SetFocus event to the movie informs the IME Manager about the new movie so subsequent IME events are sent to the newly loaded movie. Each movie can load its own candidate list, by calling SetIMEMoviePath as illustrated above and providing the path to the IME UI swf file. If a new movie doesn't call SetIMEMoviePath, the IME Manager will either use an already loaded list if one exists (possibly loaded by another movie) or not display the candidate list.

Note that developers who are targeting Korean language input without using libgfx_ime.lib do not need to create the GFx::ASIMEManager instance. They are still required, however, to pass the WM_IME messages to GFx::Movie::HandleEvent as described on the next page.

2. Tap into system IME notifications and create GFx::IMEEvents based on message parameters, passing them to the currently active movie view. For IME processing to work, the users may need to modify both the application message loop and the message processing function. The application message loop needs to be modified only if we are using the GFx::ASIMEManager for Chinese and Japanese support. If Korean support is being used without the IME manager, changes need to be made to only the message processing function.

   The application message loop is modified to allow Scaleform to receive the required keyboard messages before message translation. An example of such modification is illustrated by the following code.

```
// Grab message queue events and pass some of them to IME.
// This is not required for Korean IME without libgfx_ime.lib.
MSG msg;

if (PeekMessageW(&msg, 0, 0, 0, PM_REMOVE))
```

```
{

    // Pass raw system messages to GFxIME for pre-processing.
    if ((msg.message == WM_KEYDOWN) || (msg.message == WM_KEYUP) ||
        ImmIsUIMessage(NULL, msg.message, msg.wParam, msg.lParam))
    {
        GFx::IMEWin32Event ev(
            GFx::IMEEvent::IME_PreProcessKeyboard,
            (UPInt)msg.hwnd, msg.message, msg.wParam, msg.lParam);
        pmovie->HandleEvent(ev);
    }

    TranslateMessage(&msg);
    DispatchMessage(&msg);
}
```

The application message processing function will need to be modified to notify Scaleform about all WM_IME events. This is required for both GFx::ASIMEManager and Korean support and can be done by incorporating the following logic. The WM_IME_SETCONTEXT message is always discarded to prevent the system IME windows from showing up.

```
LRESULT UserWindowProc(HWND hwnd, UINT iMsg,
                       WPARAM wParam, LPARAM lParam)
{
    switch(iMsg)
    {
        // Pass windows IME messages to Scaleform.
       case WM_IME_SETCONTEXT:
            return DefWindowProc(hwnd, iMsg, wParam, 0);
       case WM_IME_STARTCOMPOSITION:
       case WM_IME_KEYDOWN:
       case WM_IME_COMPOSITION:
       case WM_IME_ENDCOMPOSITION:
       case WM_IME_NOTIFY:
       case WM_IME_CHAR:
       {
            GFx::IMEWin32Event ev(GFx::IMEWin32Event::IME_Default,
                                  (UPInt)hwnd, iMsg, wParam, lParam);
            // Do not pass IME events handled by Scaleform to
DefWindowProc.
            if (pmovie->HandleEvent(ev) &
                        GFx::Movie::HE_NoDefaultAction)
                return 0;
       }
        break;
    }
    return DefWindowProcW(hwnd, iMsg, wParam, lParam);
```

```
}
```

A more detailed explanation of the purpose of various IME messages is provided in Appendix D.

3. Handle focus changes among movie views by passing the appropriate focus messages to HandleEvent. If you only have one movie view in your game that receives input, your focus processing logic can be simplified. The details of focus handling are discussed below in subsection 6.3.

Note that the GFx::ASIMEManager needs to be instantiated only once and works across different movies loaded and unloaded during the life time of the Scaleform player. The candidate list movie clip is, however, bound to a GFx::Movie and will be re-created every time a new movie is loaded. If only the name but no path is specified for SetIMEMoviePath, the IME manager will try to load it from the directory where the currently active swf file is located.

6.3 Handling Focus Changes

This section describes focus changes within a movie and across multiple movies running on the player window. Focus change can result from user actions such as mouse clicks, and tab key as well as programmatically.

6.3.1 Focus Changes within a Movie

From the standpoint of IME, there are two types of Flash objects: editable and non-editable. Textfield is an example of editable object while non editable objects include movie clips, buttons etc. IME is only active when the focus is on an editable object. Whenever IME is active and the focus changes to an editable or non-editable text field, IME is finalized. By finalize we mean that the characters in the composition string that exist at the time of the focus change are finalized and become part of the text, and the state of IME is reset. The only exception to this rule is if the candidate list is open and the user clicks on a row of the candidate list. In this case, the composition string is changed to reflect the contents of the candidate list row on which the click occurred and focus is transferred back to the original editable object. IME is not finalized in this case.

6.3.2 Focus Changes across Movies

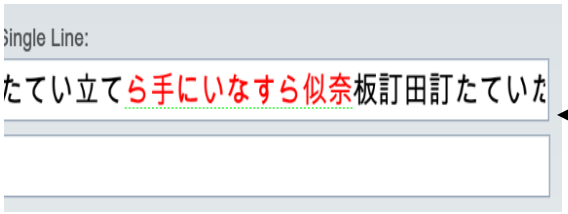
Scaleform Player allows two or more movies to be playing simultaneously on the stage. Each movie can load its own IME widgets by calling `GFx::ASIMEManager::SetIMEMoviePath` ahead of time and providing a path for the IME UI file. When focus changes from one movie to the other, the user should call HandleEvent(SetFocus) on the new movie which will finalize IME state on the previous movie and also inform IME manager about the new movie with focus.

7 Customizing the IME User Interface

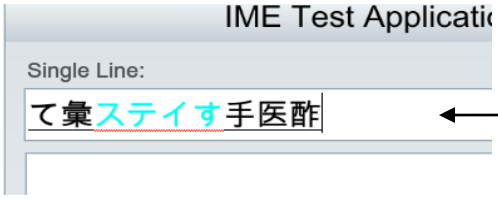
GFxIME allows for extensive customization of the appearance of the composition string, candidate list and reading window as well as setting the type of highlighting style. This section describes the parameters that can be customized and the Action Script functions used for customization.

7.1 Composition String Styles

As illustrated in the figure below, developers can customize various parameters of the composition string, clause segment, phrase length adjustment segment etc.



Composition string text in red and green underline



Clause text in light blue with dithered thick underline

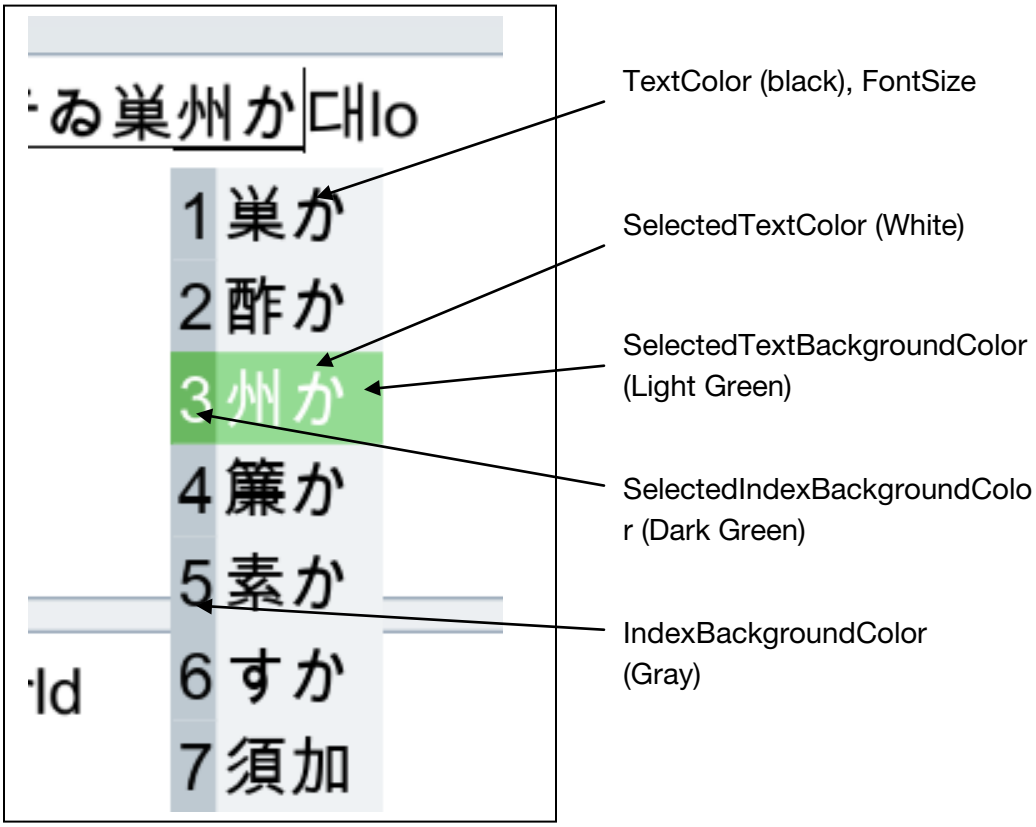
The *setIMECompositionStringStyle* function is used for changing composition string styles. An example of how to use this function is included with the IMESample.fla file. Note that we can set the styles independently for different text fields or for all the text fields at once by using TextField.prototype.setIMECompositionStringStyle as shown in the example below.

```
// Use setIMECompositionStringStyle to change the composition
// string style of Edit_SLine1.textBox text field.
var cstyle : Object   = new Object();
cstyle.textColor      = 0xff0000;
cstyle.underlineColor = 0x00ff00;
_root.Edit_SLine1.textBox.setIMECompositionStringStyle("compositionSegment",
                                                       cstyle);


// Set the style for convertedSegment for all textfields.
var clauseStyle = new Object;
```

```
clauseStyle.underlineColor = 0xFF00FF;
clauseStyle.underlineStyle = "single";
TextField.prototype.setIMECompositionStringStyle("convertedSegment", clauseStyle);
```

7.2 Candidate List Customization



The figure above displays various parts of the candidate list that can be customized.
When the candidate window or the reading window is created, we check to see if a style has previously been set on that widget and use that. If not, we use default values as the style parameter for the new widget. This is done in the Action Script functions InitCandidateListDefaults and InitReadingWindowDefaults.



Reading window text color (white) and font size

Reading window background color (dark gray)

This figure displays reading window styles that can be customized.

In order to customize some or all the UI Widget styles, the user should create a new object in AS and set the following properties.

Candidate List styles:

1. *textColor:* Font color of unselected Candidate List row text
2. *selectedTextColor*: Font color of selected Candidate List row
3. *backgroundColor*: color for the background movieclip
4. *fontSize*: candidate list row font size (both text and index)
5. *selectedTextBackgroundColor*: background color of the selected Candidate List row text
6. *indexBackgroundColor*: background color of the index background
7. *selectedIndexBackgroundColor*: background color of the index for the selected row.

Reading Window styles are similar.

Reading Window styles:

1. *readingWindowBackgroundColor*: background color of the reading window
2. *readingWindowTextColor*: text color for the reading window
3. *readingWindowFontSize*: font size for the reading window text.

The code sample below shows how to create an object, set properties on it and pass it on to the setIMECandidateListStyle function. This function is a global function and must be called from your game action script code.

```
// Create a new object to hold style.
styleObject = new Object();

// Set properties
styleObject.textColor                   = 0x5EADFF;
styleObject.selectedTextColor           = 0xFFFFFF;
styleObject.fontSize                    = 20;
styleObject.backgroundColor             = 0x001430;
styleObject.selectedTextBackgroundColor  = 0x2C5A99;
styleObject.indexBackgroundColor        = 0x12325A;
styleObject.selectedIndexBackgroundColor = 0x2C5A99;
styleObject.readingWindowTextColor      = 0xFFFFFF;
styleObject.readingWindowBackgroundColor = 0x001430;
styleObject.readingWindowFontSize       = 20;

// Pass it onto setIMECandidateListstyle
setIMECandidateListStyle(styleObject);
```

The *setIMECandidateListStyle* function sets the styles on an internal object in the IME implementation and invokes *SetCandidateListProps* and *setReadingWindowProps* functions in IME.fla with the parameters that were passed as arguments to *setIMECandidateListStyle.* The arguments that were not set are passed to *SetCandidateListProps* and *SetReadingWindowProps* as -1. This allows the user to only modify the properties that need to be changed. These functions in turn set the styles on different parts of the candidate list and reading window using movie clip, text field and button properties.

An example of how to customize candidate list parameters is presented in IMESample.fla. In this sample, the user can select among three UI themes and candidate list colors will match those on the chosen theme.

7.3 Disabling IME on specific TextFields

The user might want to disable IME on certain special purpose textfields such as the ones that take only numbers as input or the password textfield. This can be done by setting *disableIME = true* for that textfield. In the IMESample.fla file, IME is disabled on the second single line text field.

```
_root.Edit_SLine2.textBox.disableIME = true;
```

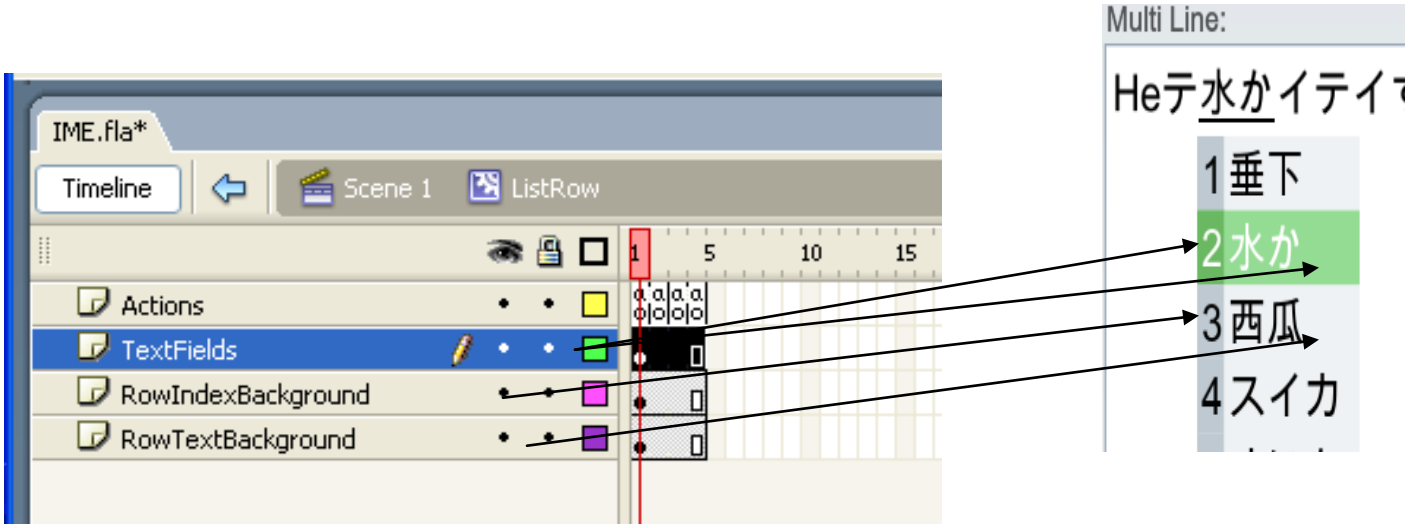
8 IME.swf UI File Structure and Customization

The two types of IME UI widgets are the candidate list and the reading window. Their implementation is distributed between IME.fla, FxCandidateListBox.as and FxCandidateListItemInterface.as files located in the Bin\IME directory. Together, these Flash files expose an API that is utilized by the C++ side of IME implementation to perform operations such as IME UI window creation, UI window positioning, item selection and so on.

The IME.fla contains the following symbols:

- ListRow
- InputWindow_mc
- InputWindowBackground_mc
- RowNumberBackground_mc
- RowTextBackground_mc
- BackgroundButton

The CandidateList is an empty movie clip containing a stack of ListRow movie clips. Each ListRow movie clip has the following structure:



The List Row Movie clip has three layers (In addition to the AS layer)

1. Layer1: Text fields

2. Layer2: Row index background movie clip (RowNumberBackground_mc)
3. Layer3: Row text background movie clip (RowTextBackground_mc)

There are two text fields in layer one. The *ID* text field is used for displaying the row index and the *Value* text field is used to display the row data (Chinese/Japanese characters).
The text background movie clip contains a Button object (symbol name: BackgroundButton) to make it responsive to mouse clicks. The text background movie clip has two keyframes: on (selected) and off (not selected) with each keyframe containing button instances of different colors so the selected row can be displayed in a different color.

The Reading window is an instance of the InputWindow_mc movie clip and uses InputWindowBackground_mc for coloring the background.

8.1 Candidate List Implementation

As mentioned before, candidate list implementation is distributed between C++ and ActionScript. Handling candidate list related messages, obtaining candidate list data such as candidate strings, current page, currently selected row etc., are performed in C++ while creating the candidate list, positioning it correctly on the stage with respect to the cursor, highlighting the currently selected row and customizing candidate list appearance is done in Action Script in the file IME.fla. This file exposes an API that is used by the C++ side of the IME implementation to interact with the UI widgets. In the following sections, we discuss the functions provided by IME.fla.

- CreateList
- DisplayInputWindow
- RemoveList
- RemoveInputWindow
- RepositionCandWindow
- RepositionInputWindow
- SetCandidateListProps
- SetReadingWindowProps

As is clear from the function names in the list above, there are three types of functionality exposed by IME.fla.

1. Creation/Destruction of Widgets
2. Repositioning the widgets
3. Setting widget properties.

1. Creation of candidate list and reading window: Candidate List is created in the CreateList function. The reading/input window is created in the DisplayInputWindow function. The reading window is much simpler than the candidate list since it only contains a text field with the reading window text and a background movie clip.

2. Repositioning the widgets: The position of the widgets is computed in C++ and passed on to Action Script where the RepositionCandWindow and RepositionInputWindow are responsible for repositioning the pop-ups correctly on the stage. For example, the candidate list must appear below the current cursor position (or above if there is not enough space below). It must also move appropriately as the user moves from one clause to the other. Note that the widgets must also be positioned correctly with respect to the boundary of the stage. If there is not enough space on the stage for the list to fit under the current line, we must check to see if there is enough space above the current line. If there is not enough space both above and below the current line, the list appears below the current line.

3. Initialization and Customization of Pop-ups: Setting the initial styles on the candidate list and reading window as well as customizing them are also handled in ActionScript. This is described in detail in a later section.

IME.fla also instantiates a FxCandidateListBox object which handles creating ListRow movie clips, obtaining candidate list data from C++, automatically resizing the various movie clips to fit contained text, highlighting the currently selected row and applying user styles on the candidate list rows.

On the C++ side, GFx::ASIMEManager implementation interprets the IME events sent to it by the movie view and calls the appropriate functions in ActionScript (AS). For the candidate list, the IME manager interacts with AS through the FxCandidateListBox C++ wrapper class, which takes care of sending candidate list data to AS, keeping track of currently visible number of rows, setting the currently selected rows etc. The candidate window/reading window related work performed in C++ is enumerated below:

1. Obtain candidate list data: This includes candidate string data, number of candidate list strings, number of rows in a page, number of pages, currently visible page, currently selected row etc from the IME context by using IMM functions.

2. Invoke CreateList: If a new list is to be opened, invoke CreateList in AS to create a new candidate list. If a list already exists, but the display needs to be modified (user changed the current page or currently selected row) this does not need to be done.

3. Positioning the UI Windows: The UI Windows must be displayed at the correct location on the stage with respect to the composition string. This is usually near the current carat position or in case of the candidate window, near the currently selected clause. IME implementation obtains the coordinates of the view rectangle corresponding to the current caret position and passes them onto *RepositionCandWindow* function in ActionScript which moves the candidate list to the correct location.

8.2 IMECommands

UI widgets need to report back to the containing movie view information about user events. For example, the candidate list should inform IMEManager if the user has clicked on a certain row. This is done using _global.imecommands. The imecommands are handled internally by the player and no special handler needs to be installed for them.

8.3 Font Management

In Scaleform 2.1, the IME pop-up swf files obtained their character glyphs from the imported font "FontBody". In Scaleform 2.2 and above, the pop-ups will obtain their resources from the underlying textfield automatically. In order for this to happen, the text fields in IME.fla must import the IMECandidateListFont. An example of this setup can be seen by opening IME.fla in the Flash authoring studio. This is a reserved font name which is used internally inside Scaleform and there should not be any other flash files that export this symbol.

An important point to understand is that only characteristics that are properties of a font such as font type, bold/italic style will be derived from the text field. Font size, on the other hand, is a property of the text field and will not automatically carry over. As illustrated in IMESample however, it's very easy to use the candidate list customization functions to set the appropriate font size for the IME pop-ups in Action Script.

9 Data Driven IME Registration System

This section describes a new data driven system for registering IMEs that will allow users to add IMEs they wish to support themselves, as long as certain conditions are satisfied. Using this system, it is possible to see the IMEs currently supported by GFxIME and add support for the localized version of those IMEs yourself without needing to rebuild Scaleform Player or GFxIME.

9.1 Existing System

The existing system for registering IMEs uses an internal list that has to be updated and the source code recompiled every time support for a new IME or a localized version of an already supported IME is added. This means that a customer has to send Scaleform a support request for every new IME since the source code for GFxIME is normally not released. The new data driven system will expose the list of supported IMEs in an XML format and allow customers to modify this list in order to add support for localized versions of already supported IMEs. The procedure to do this is described in detail below.

Scaleform IME is designed to support a wide variety of IMEs on Windows XP, Vista and 7. There are four major language types that are supported- Chinese Simplified, Chinese Traditional, Japanese and Korean. Each language has a number of IMEs. Some of these are “system IMEs” which means that they either come pre-installed with the operating system or are installed with other Microsoft products such as MS Office. Others are “third party IMEs” which means they are provided by third party vendors; Chinese and Japanese IME provided by Google and Sogou Pinyin being two popular examples.

While there is a Windows API for providing IME support, this API is highly fickle. The windows messages corresponding to different IMEs can arrive at different times, leading to dangling candidate lists, flickering and crashes. Investigating the behavior of each IME and accounting for its peculiarities is what makes IME implementation a particularly challenging task. We realized early on during the development of the IME product that it would not be possible for us to support all IMEs in a generic way. Therefore, in Scaleform 3.x, a system for registering IMEs was introduced. As part of this system, a list of supported IMEs is maintained internally. During IME initialization, we first obtain a list of IMEs installed on the user’s system and compare this list to the list of supported IMEs. Only IMEs that are common between the two lists would appear in the Scaleform language bar. For non-supported IMEs, we simply ignore all IME related messages and return `GFxIME_NotSupported` from the HandleEvent call.

The process of registering IMEs looks at the keys in the `"SYSTEM\\CurrentControlSet\\Control\\Keyboard Layouts"` section of the registry and uses a series of Windows API functions to obtain the list of IMEs registered on the system. However, it turns

out that the string names used to register these IMEs are localized according to the operating system language. For example, the Microsoft Pinyin New Experience Input Style IME appears as “Chinese (Simplified) - Microsoft Pinyin New Experience Input Style” on an English system and as “中文(简体) - 微软拼音新体验输入风格” on a Simplified Chinese system. Therefore, in order to ensure that the IMEs Scaleform support work on all operating system languages, we had to add the localized strings for every IME to our IME registration table. However, as the list of supported IMEs continued to grow, this list became unwieldy and hard to maintain. More ever, in order to add support for a specific IME requested by a customer, we would have to update our internal IME list and then rebuild the IME library. This meant that the customer would have to create a support ticket and then wait for the updated IME lib.

9.2 Data Driven IME Support

The data driven IME system uses the fact that the localized versions of a certain IME work identically across different systems. For example, the English version of the Microsoft Pinyin IME sends the same messages in the same sequence as the Chinese Simplified version. This means that once the peculiarities of a certain IME has been figured out on the English version of Windows, the same code will work on the Korean version, Chinese Simplified version etc.

9.3 Implementation

The ime.xml file contains the registration information for most of the IMEs supported by GFxIME. This section explains the steps needed to add support for a certain IME. For this procedure to work, the following conditions must be met:

1. Scaleform Version: Scaleform 3.3 and higher, since the data driven system was first introduced in Scaleform 3.3.
2. Operating System: Windows Vista/7. The steps for Windows XP will be outlined soon.
3. The IME that you are trying to add must already work on the English language version of the operating system. In other words, an English language entry for the corresponding IME must already exist in the ime.xml file.

Step 1:
Change the parameters passed during IME initialization as follows:

File: FxPlayer.cpp

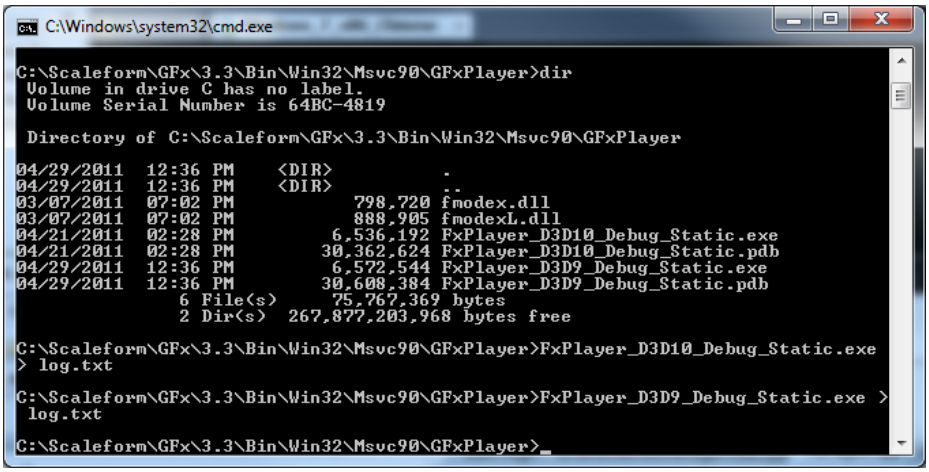
Before: `pimemanager->Init(Loader.GetLog())`

Now: `pimemanager->Init(Loader.GetLog(), pfileOpener, "C:\\Scaleform\\GFx\\4.0\\Bin\\Data\\ime.xml");`

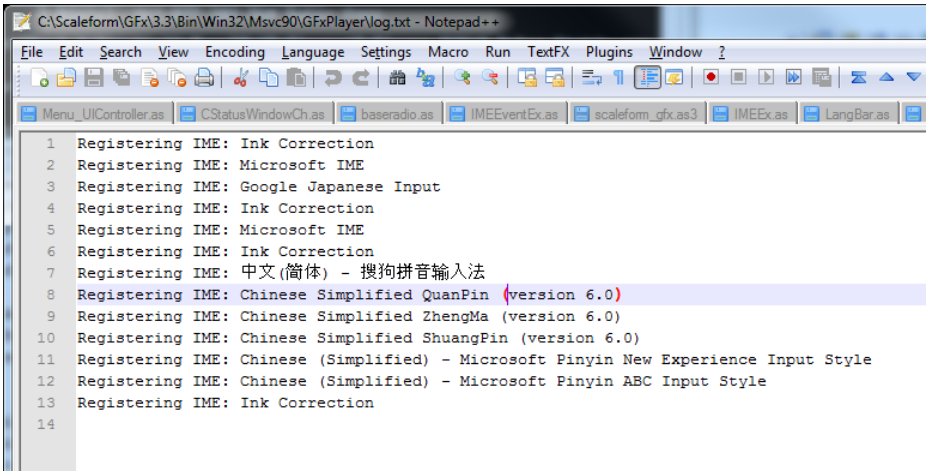
Note: You might have to change the path where ime.xml is situated on your computer.

Step 2:

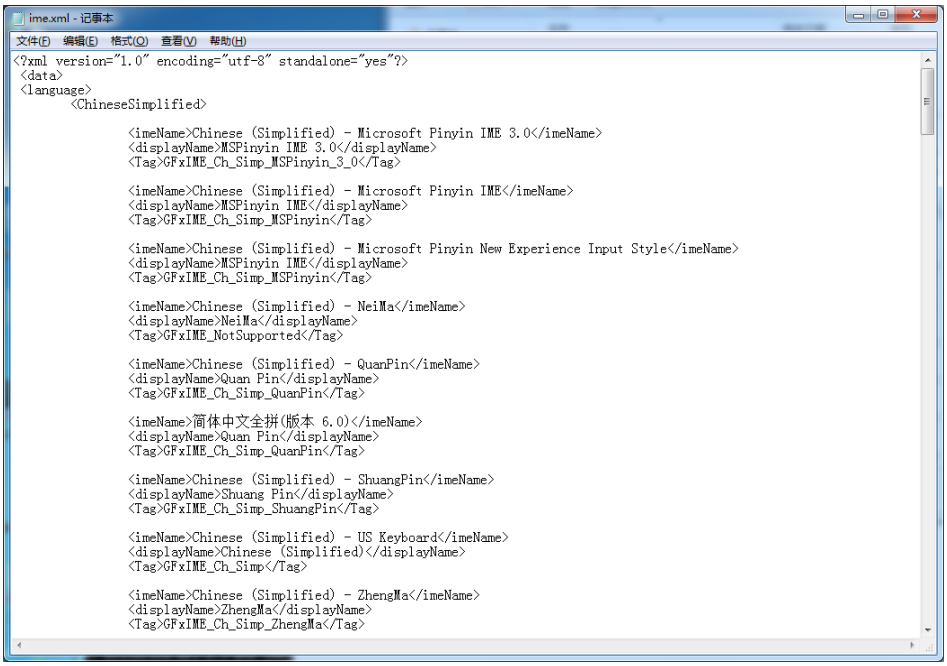
Run Scaleform Player from the console and direct output to a text file. For example:



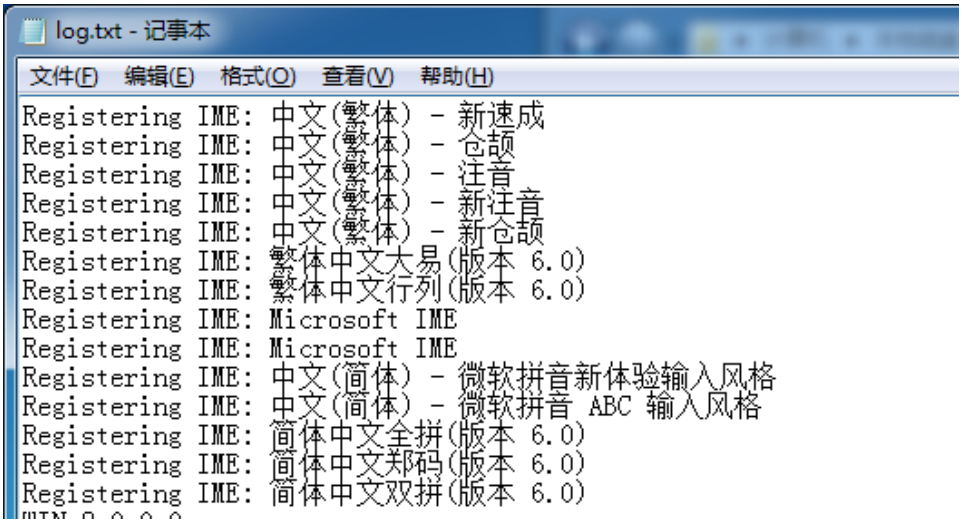
Now open log.txt. It should look like this:



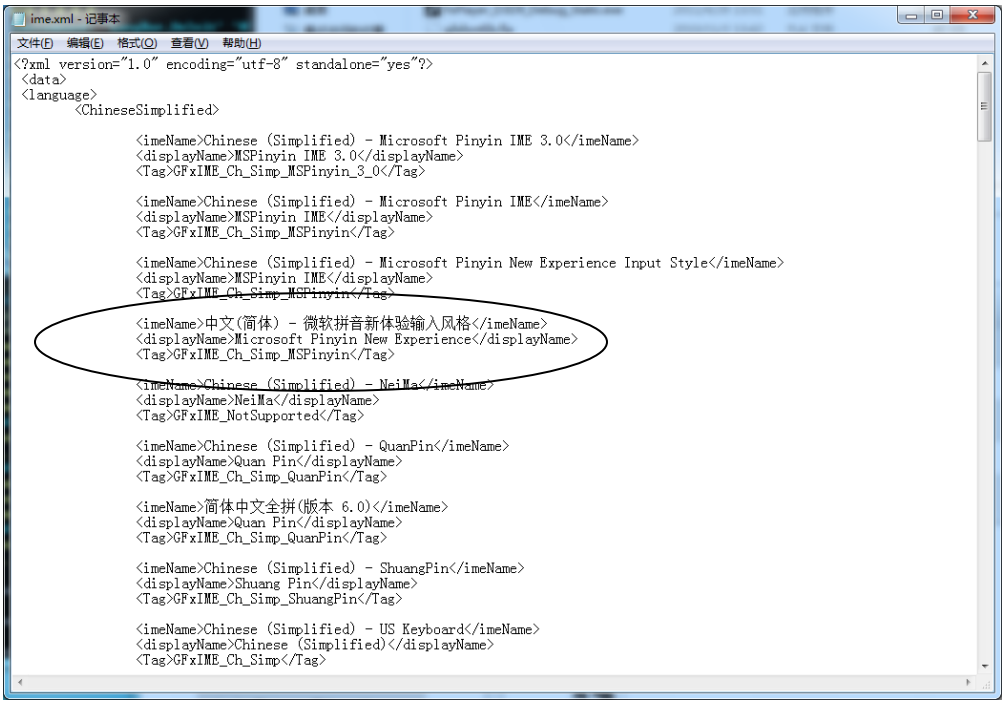
As you can see, since I'm working on an English computer, most of the IME names appear in English. Open ime.xml in a good text editor (I recommend notepad++ since it handles Unicode characters well). It should like the following:



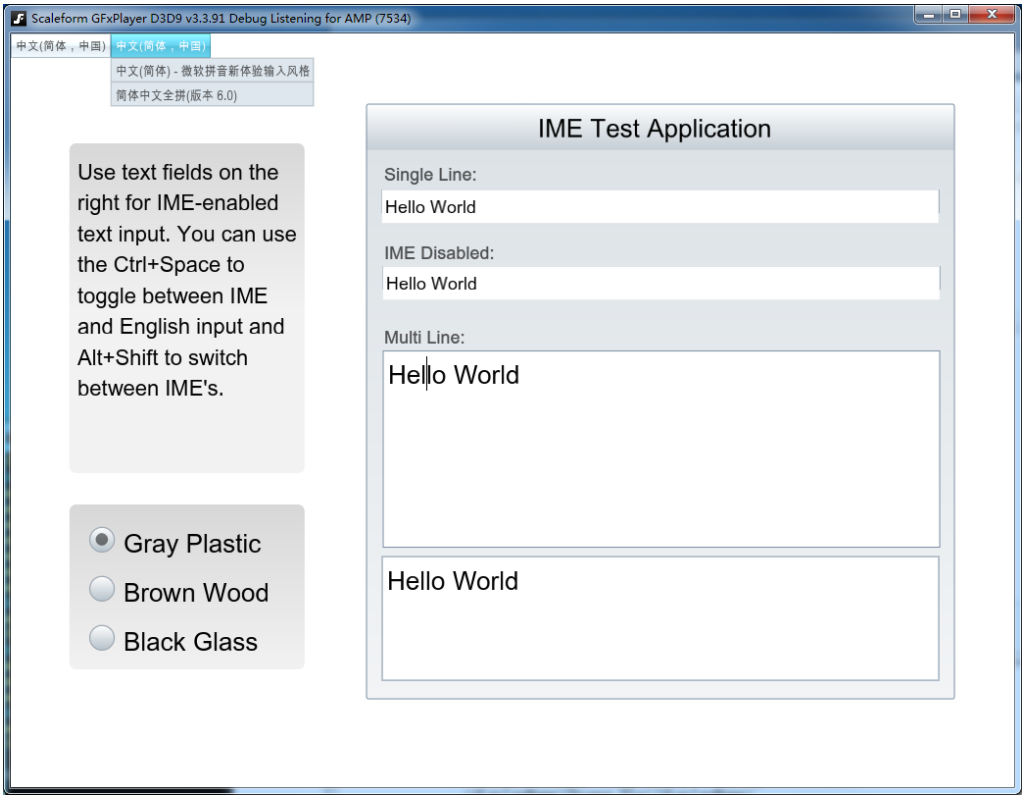
Now let's say you are working on a Chinese Simplified System, and want to add support for the Microsoft Pinyin New Experience Input Style IME. You should first run the Scaleform Player executable with its output directed to a log file as shown above. You will see something similar to the following (depending on which IMEs are installed on your system):

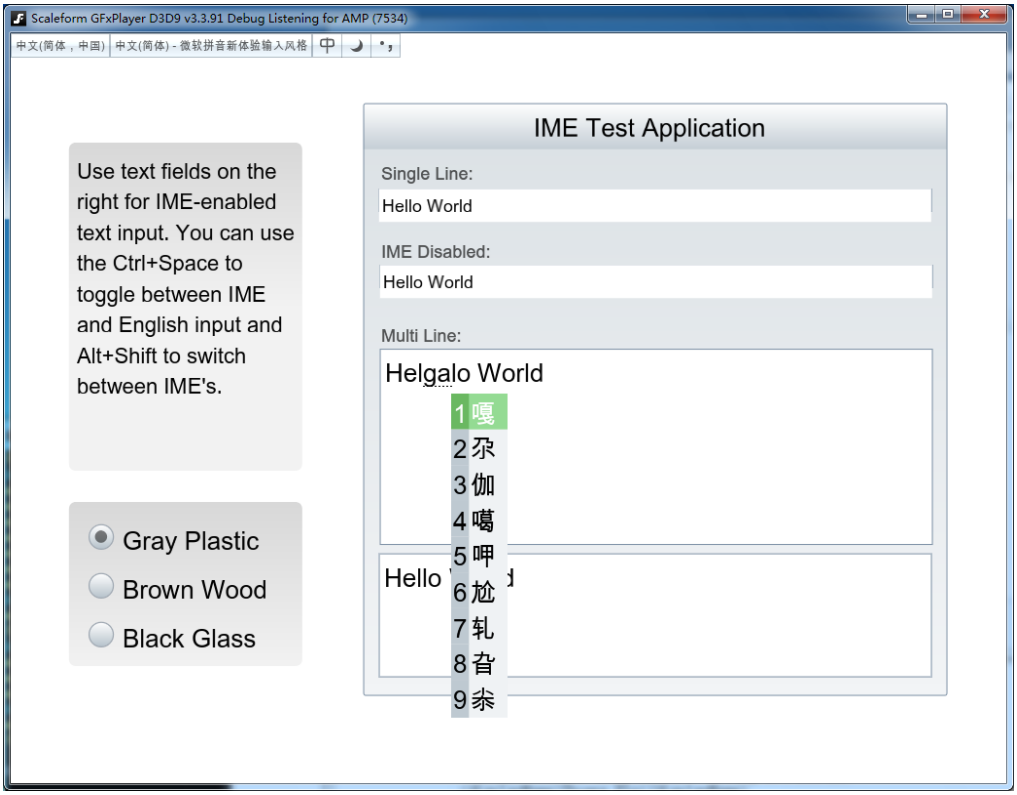


As you can see, the Pinyin New Experience IME appears in its localized form. Now in order to add support for this IME, cut and paste the string name corresponding to the Pinyin New Experience IME to ime.xml, as shown below.



Make sure to keep the Tag entry the same as the existing entry for the English system. The tag is used to identify the IME internally by GFxIME. Now, the New Experience Pinyin IME should work on the Chinese Simplified system.





That’s it! Let us know if this procedure works for you or if you encounter any difficulties.

10 Scaleform Support for Flash IME Class

This section describes implementation of Flash IME class on Scaleform.

10.1 IME support on Flash

Event summary

Event	Description
onIMEComposition = function([readingString:String]) {}	Notified when the IME composition string is being set.
onSwitchLanguage = function([languageString:String]) {}	Notified when the input language is changed using the language bar or keyboard shortcuts. This listener is not supported by Flash.

Method summary

Method Name	Description
addListener(listener:Object) : Void	Registers an object to receive notification when an IME event handler is invoked by the onIMEComposition event.
getConversionMode() : String	Indicates the conversion mode of the current IME.
getEnabled() : Boolean	Indicates whether the system IME is enabled.
removeListener(listener:Object) : Boolean	Removes a listener object that was previously registered to an IME instance with IME.addListener().
setCompositionString(composition:String) : Boolean	Sets the IME composition string.
setConversionMode(mode:String) : Boolean	Sets the conversion mode of the current IME.
setEnabled(enabled:Boolean) : Boolean	Enables or disables the system IME.

10.2 Properties

The following properties are used with the get/setConversion mode functions:

- ALPHANUMERIC_FULL
- ALPHANUMERIC_HALF
- CHINESE
- JAPANESE_HIRAGANA
- JAPANESE_KATAKANA_FULL
- JAPANESE_KATAKANA_HALF
- KOREAN
- UNKNOWN

10.3 General Observations

This section describes the language and operating system specific differences between the output of the functions listed above.

10.3.1 get/setEnabled and get/setConversionMode

10.3.1.1 *Japanese*

- Using setEnabled() can turn IME on/off. If the IME is turned off, GetEnabled() returns false.

- The IME state can be over ridden by using the language bar or setting the conversion modes using SetConversionMode(). For example, if the IME state is off, the user can use SetConversionMode() to turn IME on and set the conversion mode.

- The possible conversion modes are: : HIRAGANA, JAPANESE_KATAKANA_FULL, JAPANESE_KATAKANA_HALF, ALPHANUMERIC_HALF, ALPHANUMERIC_FULL. If any of these conversion modes are used, SetConversionMode() will return true and the state of the IME queried using GetEnabled() is true.

- If any of the other conversion modes are used, the state of the IME is not reset. For example, if the user tries to set the conversion mode to Chinese, SetConversionMode will fail and state of the IME will not be reset.

- If the user turns IME off using SetEnabled() and queries for the conversion mode, the last set conversion mode is returned.

There is also a difference in the language bar user interface between Japanese IME 2002 and 2007. In Japanese IME 2002, turning the IME off sets the conversion mode to “Direct Input”. In Japanese IME 2007, there is no “Direct Input” entry in the conversion mode list and turning IME off sets conversion mode to “Half Width Alphanumeric”. This is indistinguishable from setting the conversion mode to “Half Width Alphanumeric” except in the IME state. When the conversion mode is set to “Half Width Alphanumeric”., the IME state is on, while it’s off when IME is disabled by using setEnabled(false). However, the appearance of the language bar is the same for the two cases.

Also note that if the IME state is off and the user sets the conversion mode to “Half Width Alphanumeric” by using setConversionMode(), querying for IME state using getEnabled() will return true. However, if the conversion mode is set using the language bar, the IME state would be false.

Querying for conversion mode using GetConversionMode returns the last active conversion mode. For example, if current IME mode is HIRAGANA, and the user calls SetEnabled(false) and then obtains the conversion mode using GetConversionMode(), HIRAGANA will be returned.

10.3.1.2 Korean

- Using SetEnabled() has no effect on the IME state. The user can turn IME on/off using the language bar. GetEnabled() returns IME state as set through the language bar or by using SetConversionMode().

- The user can choose between ALPHANUMERIC_HALF, ALPHANUMERIC_FULL and KOREAN conversion modes. IME state is true for ALPHANUMERIC_FULL and KOREAN modes and false for ALPHANUMERIC_HALF. Setting the conversion mode to any of other conversion modes fails.

- GetConversionMode() returns the last set conversion mode regardless of the IME state.

Note that on Windows Vista, behaviour of Flash IME is slightly different from Windows XP. On Win XP, getEnabled() returns the IME state as set by using the language bar or setConversionMode(). Using setEnabled() to set IME state has no effect on the return value of getEnabled().

On Windows Vista, getEnabled() responds to setting IME state on/off through setEnabled(), but it doesn't have any real effect on the language bar display or the output of the IME. For example, if one is in KOREAN mode and turns IME off by calling setEnabled(false), using getEnabled() will return false, but the behaviour of the IME will not change and Korean characters are still produced.

10.3.1.3 Chinese

- setEnabled() can be used to turn IME on/off. If the IME is turned off, GetEnabled() returns false.

- The IME state can be over ridden by using the language bar or setting the conversion mode using SetConversionMode().

- The possible conversion modes are: ALPHANUMERIC_HALF, ALPHANUMERIC_FULL and CHINESE. If any of these conversion modes are used, the state of the IME queried using GetEnabled() is true.

- If any of the other conversion modes are used, the state of the IME is not reset. For example, if the user tries to set the conversion mode to Hiragana, SetConversionMode will fail and state of the IME will not be reset.

- If the user turns IME off using SetEnabled() and queries for the conversion mode, the last set conversion mode is returned.

10.3.2 OnIMEComposition Listener

This listener is invoked whenever a composition is finalized and returns the finalized string.
In Flash IME, the listener is called only for Japanese IME. For Korean and Chinese IME's the Listener is never invoked.

With our implementation, the listener is invoked for all IME's.

10.3.3 OnSwitchLanguage Listener

This listener is invoked whenever the input language is changed using the language bar or key board shortcuts. It returns a string indicating the new input language. For languages other than English, Chinese, Korean and Japanese, UNKNOWN is returned. Note that this listener only provides the name of the input language, not the specific IME. This listener is not supported by Flash.

10.4 Using disableIME *vs. setEnabled()*

It's important to understand the difference between disabling IME on a text field using the disableIME property and setting the IME state using setEnabled() method. disableIME is a property of a text field and applies to all input methods used on the text field while setEnabled() sets the system IME on/off and is not specific to any textfield. For example, if the user sets disableIME to true for a textfield, IME will always be disabled for that textfield regardless of the state of the system IME. On the other hand, if the user turns IME off for a text field using setEnabled(false) and then moves to a different text field and turns IME on, IME will stay turned on when the user transfers focus back to the first text field.

Hence, use disableIME for text fields such as password fields or fields that only take numbers as input and never expect text that need IME. Use setEnabled() when you want to disable IME temporarily.

11 Function Reference

11.1 setIMECompositionStringStyle () method

```
public function setIMECompositionStringStyle(categoryName:String,
                                    styleDescriptor:Object): Number
```

Player version: 2.1.50

Sets the style to the appropriate category of IME color settings. The `categoryName` parameter may contain the following values:

`"compositionSegment"` – sets color settings for the whole composition string;
`"clauseSegment"` – sets color settings for clause segment;
`"convertedSegment"` – sets color settings for converted text segment;
`"phraseLengthAdj"` – sets color settings for phrase length adjustment;
`"lowConfSegment"` – sets color settings for low-confidence characters.

The `styleDescriptor` is an Object instance with the following optional members set:

`textColor : Number`
The color of text.

`backgroundColor : Number`
The background color.

`underlineColor : Number`
The color of underline.

`underlineStyle : String`
The style of underline. Valid values:

```
"single"
"thick"
"dotted"
"itheredSingle"
"ditheredThick"
```

All colors are specified in the following format: 0xRRGGBB, where RR – hexadecimal representation of red component [0..255], BB – hexadecimal representation of blue component [0..255], GG – hexadecimal representation of green component [0..255].

11.2 setIMECandidateListStyle () method

```
public function setIMECandidateListStyle(styleDescriptor:Object)
```

Player version: 2.1.50

Sets the candidate list style to that of the object passed as argument.

The `styleDescriptor` is an Object instance with the following optional members set:

```
styleObject.textColor                    = 0x5EADFF;
styleObject.selectedTextColor            = 0xFFFFFF;
styleObject.fontSize                     = 20;
styleObject.backgroundColor              = 0x001430;
styleObject.selectedTextBackgroundColor  = 0x2C5A99;
styleObject.indexBackgroundColor         = 0x12325A;
styleObject.selectedIndexBackgroundColor = 0x2C5A99;
styleObject.readingWindowTextColor       = 0xFFFFFF;
styleObject.readingWindowBackgroundColor = 0x001430;
styleObject.readingWindowFontSize        = 20;
```

`textColor : Number`
: The color of text.

`selectedTextColor : Number`
: The color of selected text.

`fontSize : Number`
: font size of the text in the row and row index.

`backgroundColor : Number`
: The background color of the text portion of unselected rows.

`seletedTextBackgroundColor : Number`
: The color of the text portion of the selected row.

`indexBackgroundColor : Number`
: The background color of the index portion of unselected rows.

`selectedIndexBackgroundColor : Number`

The background color of the index portion of selected rows.

`readingWindowTextColor : Number`
The color of the text in the reading window.

`readingWindowBackgroundColor : Number`
Background color of the reading window.

`readingWindowFontSize : Number`
Font size of the text in the reading window.
All colors are specified in the following format: 0xRRGGBB, where RR – hexadecimal representation of red component [0..255], BB – hexadecimal representation of blue component [0..255], GG – hexadecimal representation of green component [0..255].

11.3 getIMECompositionStringStyle () method

```
public function getIMECompositionStringStyle(categoryName:String): Object
```

Player version: 2.1.50

Returns the color settings descriptor object for the specified category of IME color settings. See `setIMECompositionStringStyle` for detailed description of the `categoryName` and the descriptor object.

Parameters
`categoryName:String` – Category of IME color settings (see `setIMECompositionStringStyle`).

Returns
Object - The descriptor of color settings for the category (see `setIMECompositionStringStyle`).

See also:
`setIMECompositionStringStyle`
`disableIME`

11.4 getIMECandidateListStyle () method

```
public function getIMECompositionStringStyle(categoryName:String): Object
```

Player version: 2.1.50

Returns the color settings descriptor object for IME candidate list. See `setIMECandidateListStyle` for detailed description of the descriptor object.

Returns

Object - The descriptor of color settings for the candidate list.

See also:

`setIMECandidateListStyle`
`disableIME`

12 Appendix A: Language Bar and Status Window in Scaleform

The following section describes the implementation of language bar and status window in Scaleform on Win32. Scaleform 2.2 and older versions do not have these features and users can skip this section if not using Scaleform 3.0 and higher versions.

Scaleform 3.0 introduces two new IME features- the Language Bar that allows the user to change the current input language and IME and the Status Window for changing the conversion mode of the currently active IME. The addition of these to the previously available Input Method Editor (IME) features – composition string, candidate window and the reading window provides a complete IME UI solution, allowing the user to switch between input languages and type in East Asian language text in a full screen mode game with the same comfort and ease of use as in a windowed application.

The Language bar is a floating toolbar that appears on your desktop automatically when you add handwriting recognition, speech recognition, or an IME as a method editor. The buttons and options that are displayed on the language bar depend on the text services you have installed, and on the program that is currently active. For example, Microsoft Word 2003 supports speech recognition, while Notepad does not. If both programs are running, the speech buttons are displayed when Word is active, but disappear when Notepad is made the active program. Similarly, if the user adds a second language or keyboard layout, it can be selected from the list of installed languages on the language bar.

Note that the Windows language bar can be used to turn on text services such as writing pad and speech recognition in addition to switching the input language. Scaleform implementation of the language bar is restricted to be used for the purpose of changing the current input language or IME.

Status window is a small interactive window that conveys the conversion mode state of the currently used IME. The information provided by the status window and its shape and location can vary widely depending on the IME being used. The user can usually interact with the status window by clicking on it to change the conversion mode of the IME. For Chinese IME's, the status window usually appears on the bottom left part of the screen while for Japanese and Korean, the status window usually appears integrated with the language bar.

12.1 Using the System Language Bar and Status Window

This section describes how to the use the system Language bar to change the current input language and IME and the status window to change the conversion mode of the currently active IME.

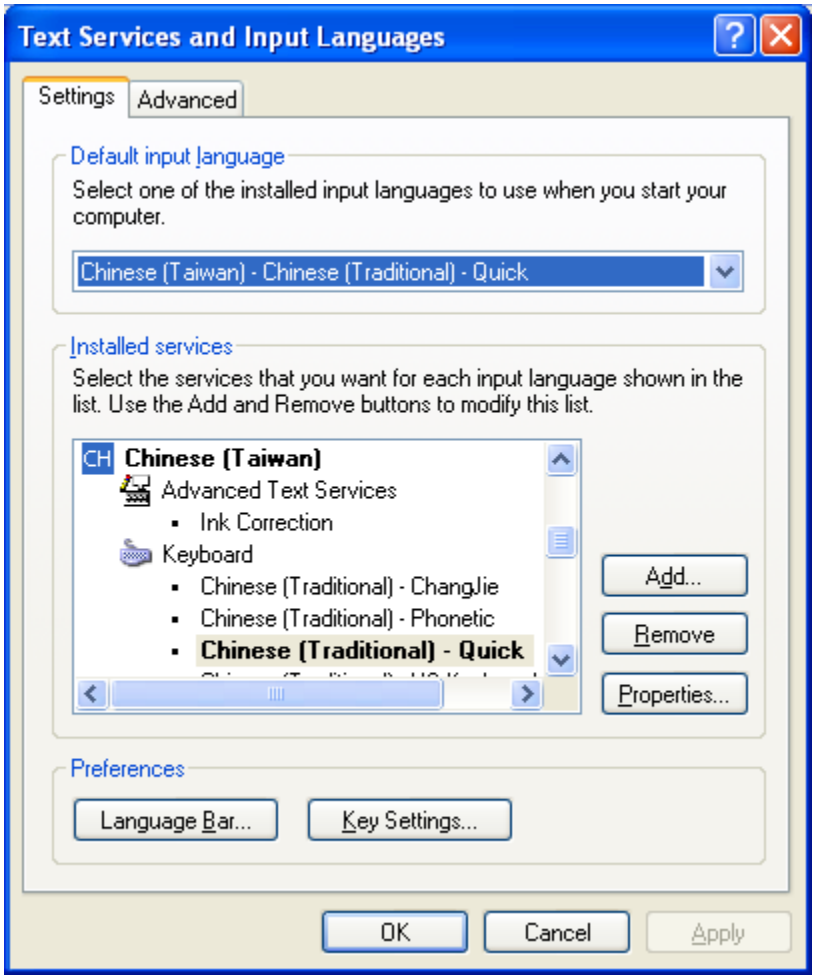


The diagram above shows the two main components of the language bar. The left tab displays the current input language while the right tab displays the currently active IME for that language. The user can click on these tabs to see the list of installed input languages or IMEs corresponding to a certain input language.





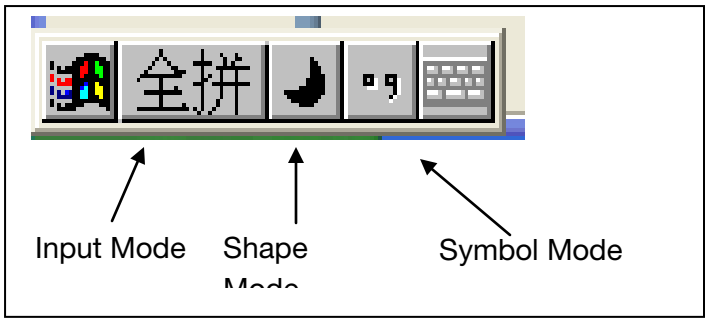
Not all input languages have associated IME's. For example, selecting the English input language causes the IME tab to disappear on the language bar. The user can use the **Text Services and Input Languages** dialog box (accessible from the Control Panel) to add and remove input languages and IME's. An illustration of this dialog box on Windows XP is shown below:



The language bar can also provide support for more advanced IME features such as soft keyboard, dictionary pad etc. Our implementation of the language bar is limited to displaying a list of installed input languages and the IME's corresponding to each language.

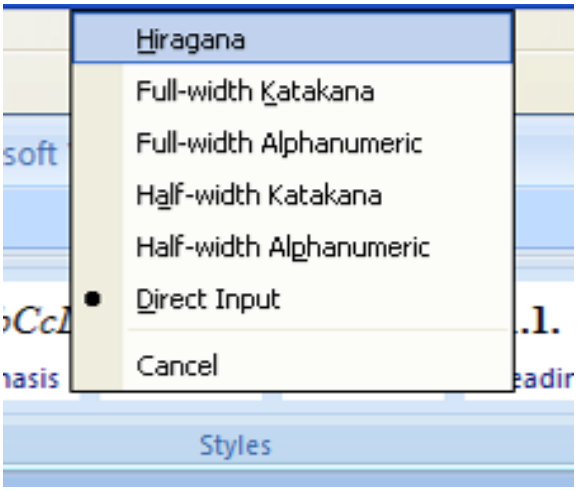
The status window displays the current conversion mode and allows the user to toggle between different components of the conversion mode. The appearance of the status window differs greatly across different input languages.

The graphic below shows a typical Chinese status window along with its three main components.



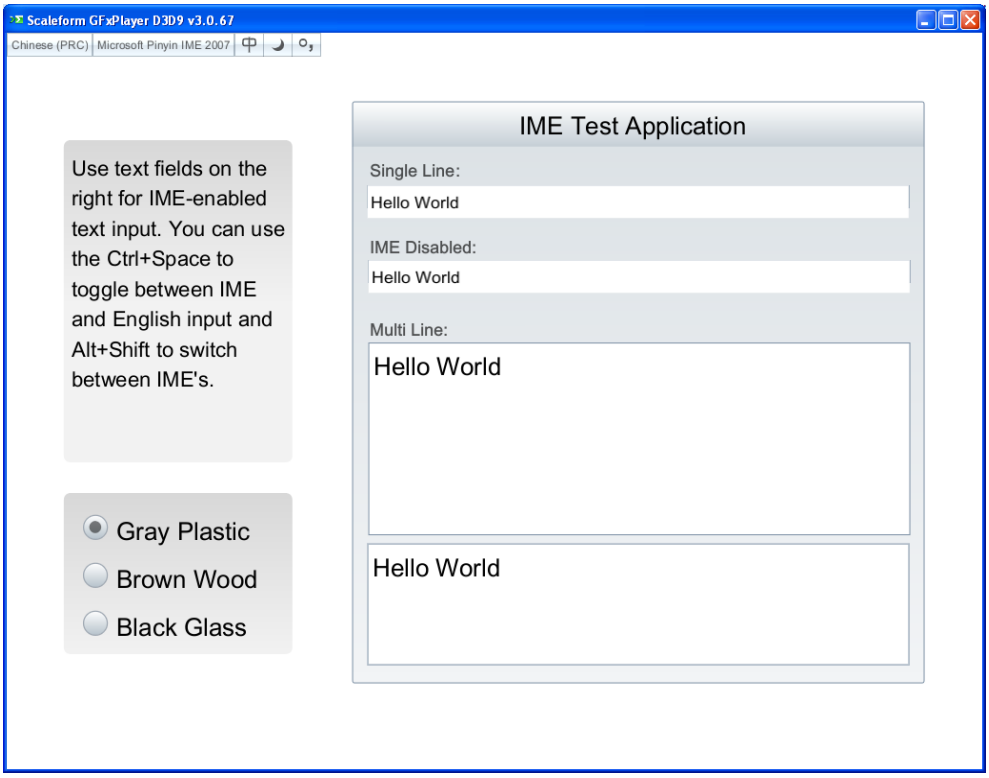
- **Input Mode:** This can be used to toggle between English input (English mode) and Asian language input (Native mode). In the Native mode, the keystrokes and converted by the IME layer into Asian characters and the appropriate IME messages are generated. In English mode, no conversion is performed and characters are output in response to keystrokes as if the input language was English.
- **Shape Mode:** This can be used to toggle between producing half width and full width characters in English mode. It has no effect in Native mode.
- **Symbol Mode:** This can be used to toggle between producing different type of punctuation symbols. Like Shape mode, this mode is only effective in English mode.

The Japanese language status window is integrated with the language bar and provides a list of 6 different conversion modes:



12.2 Language Bar Demo Shipped With Scaleform 3.x+

Upon installing IME for Scaleform 3.x, you can use the IMESample.swf file from the *bin\data\{AS2 or AS3}\ime* directory to see an example of the language bar and the core IME functionality working together. When you run this file in Scaleform Player, you should see a display resembling the following (the input language and the IME shown will depend on your system settings):



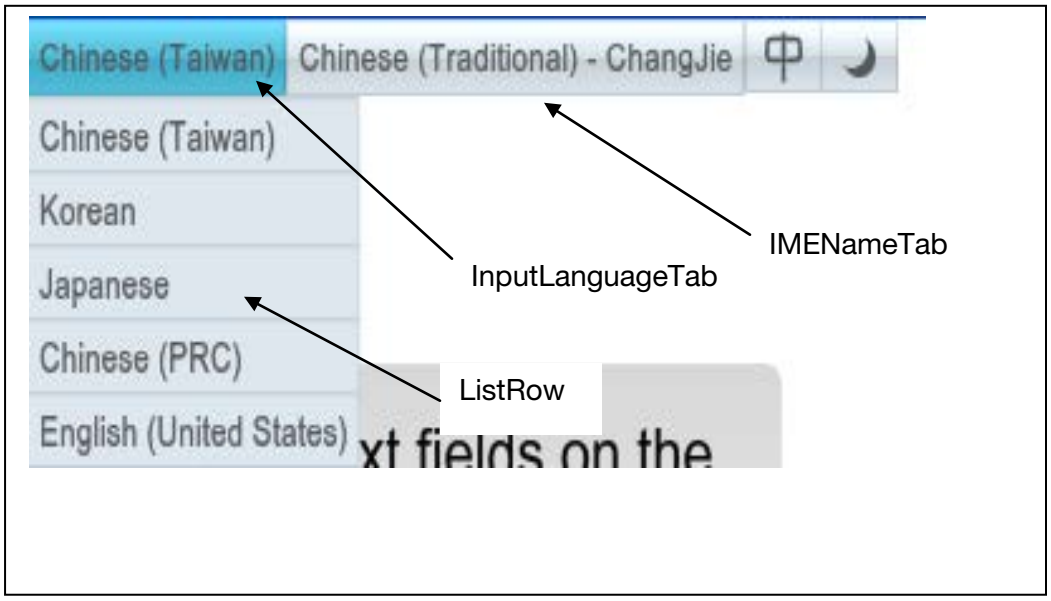
You can click on the input language tab and the IME tab to change the input language or the IME. An important point to remember is that the Scaleform language bar will display all the languages installed on your system in the input language list, but only the IME's that are supported by GFxIME. So if you are using a non-supported IME, it will not appear in the list of IME's.
The position of the language bar and the color of its text and background will change when you click on the "Brown Wood" and "Black Glass" buttons. This is to demonstrate customization of the language bar display to match the current UI theme. Also, in the "Brown Wood" theme, the language bar will appear with a grip which can be used to drag the language bar around the Scaleform Player window.

12.3 Implementation of the Language Bar and Status Window in Scaleform

The implementation of the Language bar and the Status window is distributed between GFxIME core and ActionScript. Obtaining system specific information such as list of installed input languages, IME's and getting and setting conversion mode information is done in C++ inside GFxIME. Creating and managing the movie clips that constitute the language bar and the status window are done in Action Script. Flash movies can install listeners to receive notifications for Language bar and Status Window related events and use "imecommands" to send information about user interaction to GFxIME.

Language Bar

Scaleform IME Language bar is implemented in the flash file "LangBar.fla" and consists of the following parts:



- **InputLanguageTab:** The user can click on this tab to see a list of input languages installed on the system that are supported by GFxIME.
- **IMENameTab:** When the user clicks on an input language in the input language list, the name of the last used IME corresponding to this input language is displayed in the IMENameTab. The user can also click on the IMENameTab to see a list of installed IME's corresponding to the current input language.

Most of the implementation of the language bar takes place in ActionScript. There are three main aspects of the implementation.

- **Maintenance of the appearance of the Language bar:** This includes resizing the movie clips to fit the contents of the text contained, repositioning the movie clips so that the IMENameTab always stays to the right of the InputLanguageTab, changing the color of the text and the background to match the UI theme etc. This is done using standard ActionScript constructs and can be easily understood by going through the code in the language bar flash files.
- **Obtaining the Language bar contents:** This involves obtaining the current language bar state (comprising of the input language, IME and conversion mode) when the language bar is initialized and receiving notifications when the state changes due to user actions. This is achieved by installing listeners for language bar events enumerated below.
- **Informing GFxIME about user interactions:** When the user interacts with the language bar by clicking on the input language or IME name tab or selecting an IME from the list of installed IME's, the appropriate notification must be sent to GFxIME which in turn relays it to the system. This is achieved through "imecommands" which are a specific kind of fscommands used for IME related notifications.

This design allows for the maximum ease of use and flexibility while freeing the user from system specific considerations. GFxIME takes care of obtaining the list of supported languages and IME's from the system, changing the current input language and IME, sending and receiving status window notifications etc.

Now we proceed to describe the API induced by the imecommands and the language bar related events for which the user can install listeners for.

Language bar related IME Events

- **onSetCurrentInputLanguage**

  Parameters: String containing the new input language.
  Description: Sent when the current input language changes or when the language bar is initialized.

- **onSetSupportedLanguages**

  Parameters: String containing the installed input languages separated by "@@". This delimiter has been chosen since it's very unlikely to appear as part of an input language. Other delimiters such as comma can sometimes be part of the input language itself leading to parsing errors.
  Example: "Japanese@@Chinese, Simplified@@Korean@@English"

Description: As the name indicates, the parameter provides a list of the installed input languages. This message is received when the user clicks on the InputLanguageTab and an installed input languages list must be presented.

- **onSetIMEName**

  Parameters: String containing the name of the current IME.
  Description: This message is sent when the user selects an IME from the list of installed IME's or when a new input language is selected. There is usually a default IME associated with every input language. This default is preset by the system when the application starts and is the last chosen IME for that input language thereafter. For example, suppose the user has installed Quan Pin, Shuang Pin and Microsoft Pinyin IME's for the Chinese (Simplified) input language. Also, assume that the default IME for Chinese (simplified) is Microsoft Pinyin. When the language bar enabled Scaleform Player is started and Chinese (Simplified) is chosen as the input language for the first time, Microsoft Pinyin will be displayed in the IMENameTab. Now if the user selects a different IME (say Quan Pin) and then switches to a different input language, Quan Pin will become the default IME for Chinese (Simplified). Hence, when the user switches back to Chinese (Simplified) again, Quan Pin will be displayed in the IMENameTab.

- **onSetSupportedIMEs**

  Parameters: String containing the installed IME's for the chosen input language separated by "@@".
  Description: This is used to provide a list of the installed IME's. This message is received when the user clicks on the IMENameTab.

IMECommands:

- **LangBar_OnInit:**

  Syntax: `_global.imecommand("LangBar_OnInit", 0);`
  Description: This command should be sent from the first frame of the movie implementing the Language Bar. It causes the onSetCurrentLanguage and onSetIMEName messages to be received allowing the movieclip to initialize its state correctly.

- **LangBar_GetInstalledLanguages:**

  Syntax: `_global.imecommand("LangBar_GetInstalledLanguages", 0);`
  Description: Sent when the user clicks on the InputLanguageTab to request a list of the installed input languages

- **LangBar_OnSelectInputLang:**

Syntax: `_global.imecommand("LangBar_OnSelectInputLang", inputLangName);`
Description: Sent when the user selects an input language from the input language list. The parameter is the input language name. GFxIME causes the system input language to be set to the new language in response.

- **LangBar_GetIMENames:**

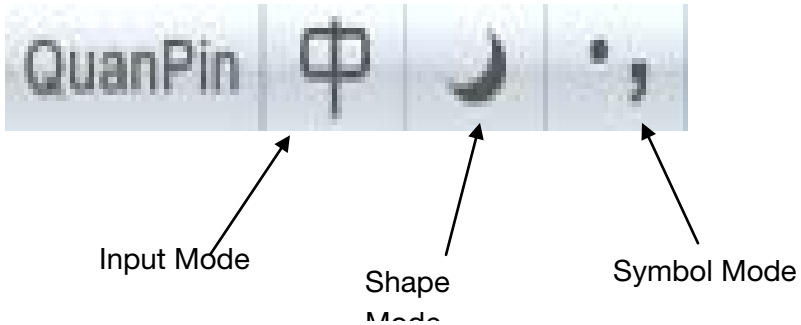
  Syntax: `_global.imecommand("LangBar_GetIMENames", inputLangName);`
  Description: Sent when the user clicks on the IMENameTab to obtain the list of installed IME's for the input language specified in the inputLangName parameter.

- **LangBar_OnSelectIME:**

  Syntax: `_global.imecommand("LangBar_OnSelectIME", imeName);`
  Description: Sent when the user selects an IME from the IME names list. The parameter is the IME name. GFxIME causes the current IME to be set to the new IME in response.

Status Window

The following describes the structure and implementation of the status window. The status window is a floating toolbar that is used to display and change the current conversion mode. The Chinese IME status window has three main components:



- **Input Mode:** Used to toggle between English mode input (keystrokes are output without any conversion by the IME layer) and Native mode (IME layer intercepts the keystrokes and IME messages are sent in response)
- **Shape Mode:** Used to toggle between half width and full width mode. This has effect only when InputMode is set to English.
- **Symbol Mode:** This is used to toggle the shape of the punctuation marks produced.

The system Chinese status window can also provide IME specific information such as an icon that represents the IME currently being used, soft keyboard etc. There is no public API for obtaining this information, and we don't provide any custom IME information in our status window. Also, none of the third party Chinese IME's (such as Google Pinyin IME, Sogou Pinyin IME) we have investigated so far provides any conversion mode information. Hence the status window is not displayed when third party IME's are being used.

In addition, on Windows, the status window can appear at a random position on the screen (this position is usually one of the screen corners, but varies with the type of IME and the application). In GFxIME, the status window always appears immediately to the right of the IMENameTab.

The Japanese status window consists of a list of conversion modes enumerated below. This list always appears integrated with the language bar.

- Hiragana
- Full-Width Katakana
- Full-Width Alphanumeric
- Half-Width Katakana
- Half-Width Alphanumeric
- Direct Input



Figure 1: Japanese Status Window

On Windows, the symbol mode button is not displayed when Chinese Traditional IME's are being used. We replicate this behavior and hide the symbol mode when Chinese Traditional IME's are used.



Figure 2: Chinese Traditional Status Window

The Korean status window consists only of the input mode. Symbol mode and Shape mode symbols are absent.



Figure 3: Korean Status Window

When the user clicks on a status window component, the system is instructed to toggle the part of the conversion mode corresponding to the clicked component. Upon successful change in the conversion mode, the system sends back a notification containing information about the new conversion mode which can be used to update the status window display.

We now list the flash files that are needed to implement the status window and briefly describe how it works. The following flash files are involved in the implementation of the status window:

- StatusWindow.fla
- StatusWindow_ch.fla
- StatusWindow_kr.fla
- StatusWindow_jp.fla
- CStatusWindowBase.as
- CStatusWindowCh.as
- CStatusWindowKr.as
- CStatusWindowJp.as

The base status window functionality is contained in StatusWindow.fla which delegates the language specific implementation to StatusWindow_ch.fla, StatusWindow_kr.fla and StatusWindow_jp.fla. Just like the language bar, the communication between the movie implementing the status window and GFxIME takes place through imecommands and status window notification listeners.

Status Window related IME Events

- **onSwitchLanguage**

  Parameters: String containing the new input language.
  Description: Sent when the current input language changes. The status window movie clips maintains an ActiveLanguage variable which points to the language specific status window implementation. This variable is reset when this message is received to point to the correct derived class.

- **onRemoveStatusWindow**

  Parameters: None.
  Description: The status window movie should be removed/hidden in response to this message.

- **onSetConversionMode**

  Parameters: String indicating the conversion mode value.
  Description: Used to set the conversion mode when the status window is initialized or when the conversion mode is changed by the user. The parameter string is a comma separated string containing the conversion mode values. As described above, the conversion mode has three components- input mode, shape mode and symbol mode. The input mode can be either “Native” or “Alphanumeric” and shape and symbol mode can be “Half” or “Full”. So a typical conversion mode string can be: “Native, Half, Full”.

- **onDisplayStatusWindow**

  Parameters: None.
  Description: Used to display the status window.

IMECommands:

- **StatusWindow_OnInit:**

  Syntax: `_global.imecommand("StatusWindow_OnInit", 0);`
  Description: This command should be sent from the first frame of the movie implementing the status window. It causes the onSetConversionMode and onDisplayStatusWindow messages to be received allowing the status window movie clip to initialize its state correctly.

- **StatusWindow_OnShape/OnInputMode/OnSymbol:**

  Syntax:
  `_global.imecommand("StatusWindow_OnInputMode", toggleState);`
  `toggleState` = true if InputMode = native; false otherwise.

`_global.imecommand("StatusWindow_OnShapeMode", toggleState);`
`toggleState` = true if ShapeMode = full; false otherwise.

`_global.imecommand("StatusWindow_OnSymbolMode", toggleState);`
`toggleState` = true if SymbolMode = full; false otherwise.

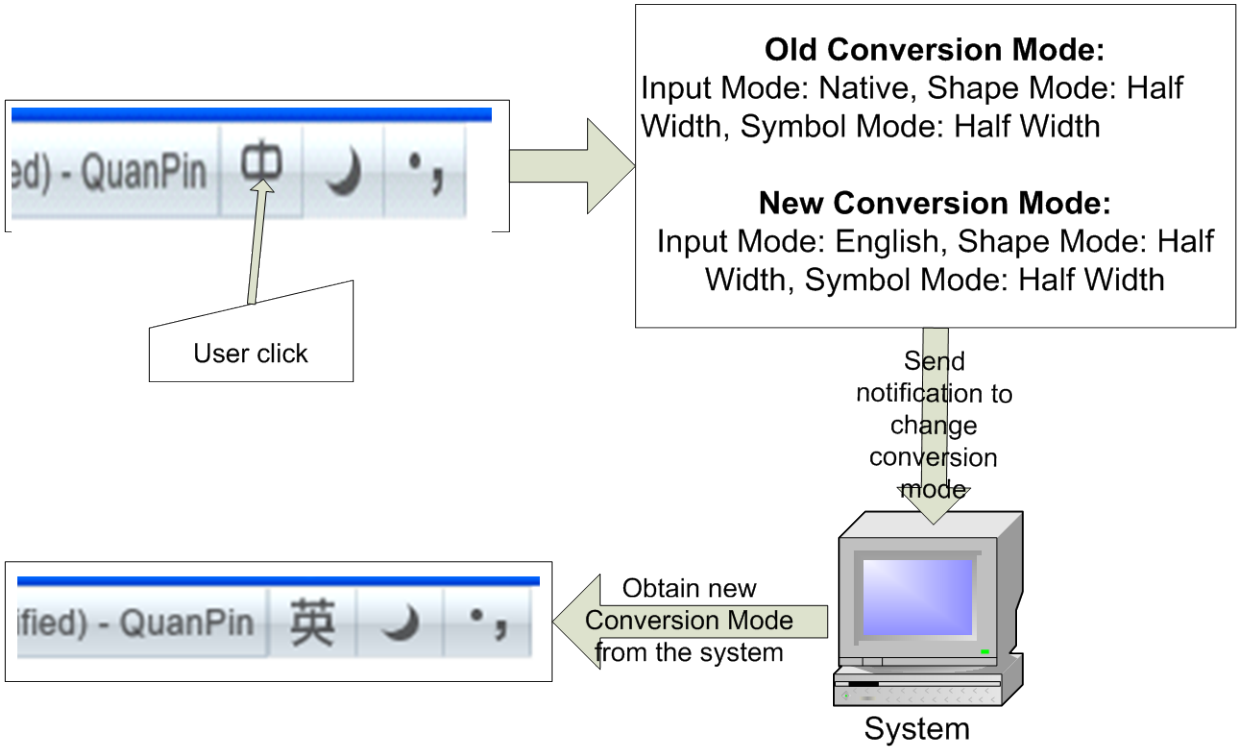
Description: Sent when the user clicks on a component of the status window. When the user clicks on the input mode tab, the "StatusWindow_OnInputMode" command is sent and so on. The toggleState reflects the current state of the conversion mode component.

When the Status Window is initialized, the following actions are performed in the first frame:

- The swf's containing the language specific implementation- StatusWindow_jp.swf, StatusWindow_kr.swf and StatusWindow_ch.swf are loaded.
- The listeners for the status window messages are installed.
- To obtain the current input language and the conversion mode, StatusWindow_OnInit imecommand is sent which causes the onSwitchLanguage and onSetConversionMode messages to be sent. These messages provide the information needed to initialize the state of the Status Window correctly.

A variable is maintained which always points to the StatusWindow implementation corresponding to the current input language. Whenever the Input language is changed, onSwitchLanguage notification is sent allowing for the StatusWindow variable to be set accordingly.

When the user clicks on a component of the StatusWindow, a "StatusWindow_OnShape/InputMode/Symbol" imecommand is sent (depending on which component of the window was clicked) along with the toggle state of that component. In response to this imecommand, GFxIME will instruct the system to change the corresponding part of the conversion mode. For example, if the current input mode is Native and the user clicks on the Input Mode tab, the imecommand "StatusWindow_OnInputMode" will be sent with the current state of the input mode (Native). In C++, the system will be instructed to change the Input mode part of the conversion mode to English. Upon successfully changing the conversion mode, GFxIME sends a onSetConversionMode message with the new conversion mode to toggle the display on the status window appropriately. The figure below outlines this process.



10.3.1 Using the Language Bar in your Game

Any movie can be used to implement the language bar as long as it installs the listeners listed above and sends the appropriate imecommands. This architecture also provides the user the ability to implement a subset of the language bar functionality. For example, if the user wants to just display the current input language, they only need to install a listener for “onSwitchLanguage” message.

13 Appendix B: IME Screenshots

This section presents screenshots of supported IME's on various test platforms.

13.1 Chinese (Simplified) Vista

13.1.1 Simplified Chinese IMEs



Comments:

- Wubi86, Wubi98, Google Pinyin, Sogou Pinyin IME’s were installed separately as third party IME’s.
- ABC IME v5.30 does not work correctly.
- ABC input mode of the MSPinyin IME 2003 and 2007 does not work correctly (even on Notepad).

13.1.2 Traditional Chinese IMEs



13.2 Chinese (Traditional) Vista

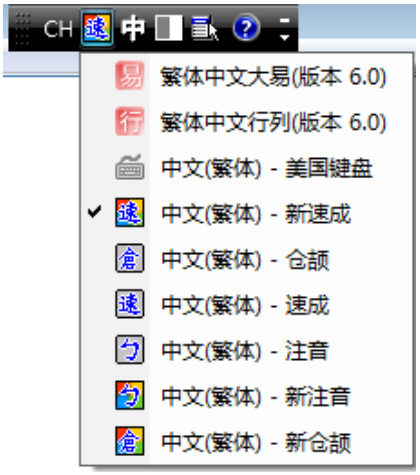
13.2.1 Simplified Chinese IMEs



Comments:

- ABC IME candidate list does not work properly.

13.2.2 Traditional Chinese IMEs

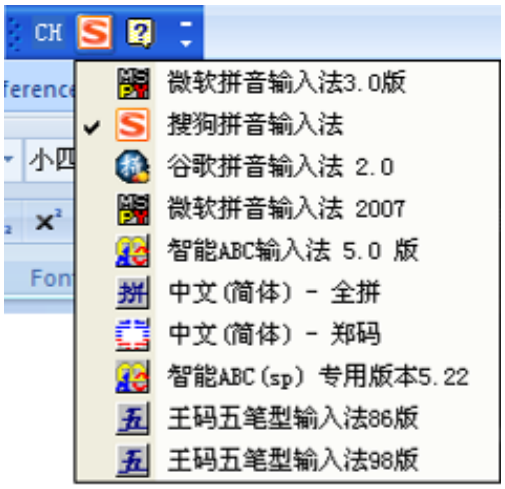


Comments:

- Clicking on secondary candidate lists (the ones that need Shift + Number key to select candidate list row) does not work. Rows can always be selected using the Shift + Number key technique.

13.3 Chinese (Simplified) XP

13.3.1 Simplified Chinese IMEs



Comments:

- ABC 5.22 does not work correctly, ABC 5.0 works ok.

13.3.2 Traditional Chinese IMEs

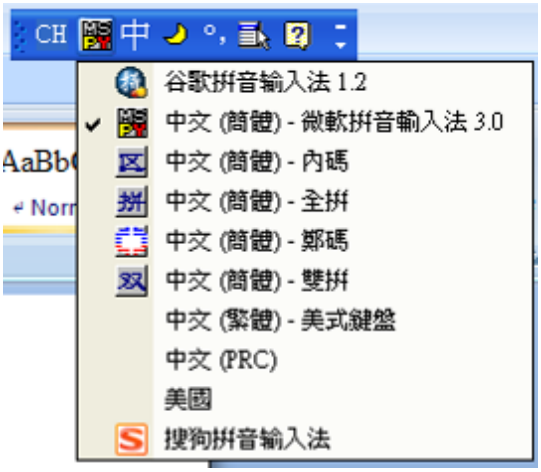


Comments:

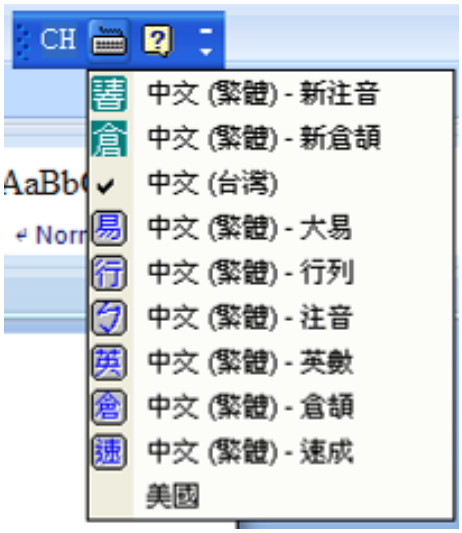
- New Changjie reading window does not work correctly unless the Microsoft Office IME 2003 is installed (http://www.microsoft.com/downloads/details.aspx?familyid=f2a55210-4cc1-4040-b710-625a45f2594d&displaylang=en).

13.4 Chinese (Traditional) XP

13.4.1 Simplified Chinese IMEs

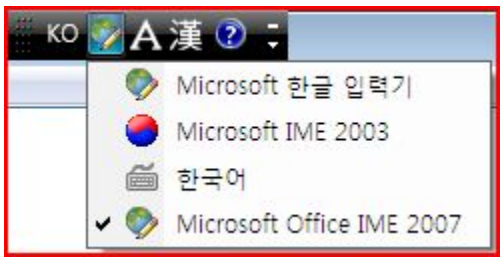


13.4.2 Traditional Chinese IMEs



13.5 Korean IMEs

13.5.1 Korean Vista



13.5.2 Korean XP



13.6 Japanese IMEs

In Japanese XP and Vista, installing Japanese IME 2007 overwrites Japanese IME 2003 and installing ATOK 2009 overwrites ATOK 2008. Therefore, screenshots of the language bar before and after installing Japanese IME 2007 and ATOK 2009 are presented.

13.6.1 Japanese XP

Before ATOK2009 and Japanese IME 2007 were installed:

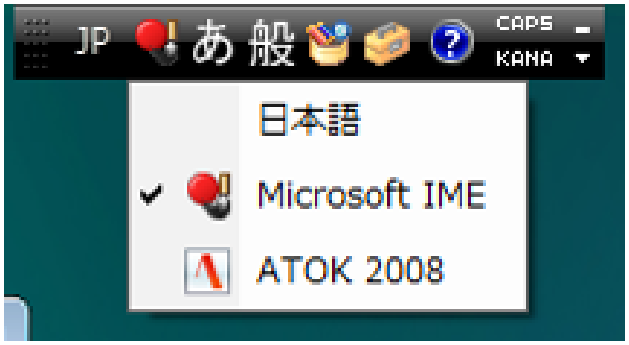


After ATOK 2009 and Japanese IME 2007 were installed:



13.6.2 Japanese Vista

A clean install of Japanese Vista comes with IME 2003. After installing ATOK 2008, language bar looks like:



After ATOK 2009 and Office 2007 are installed (ATOK 2008 and MS IME 2003) get overwritten.



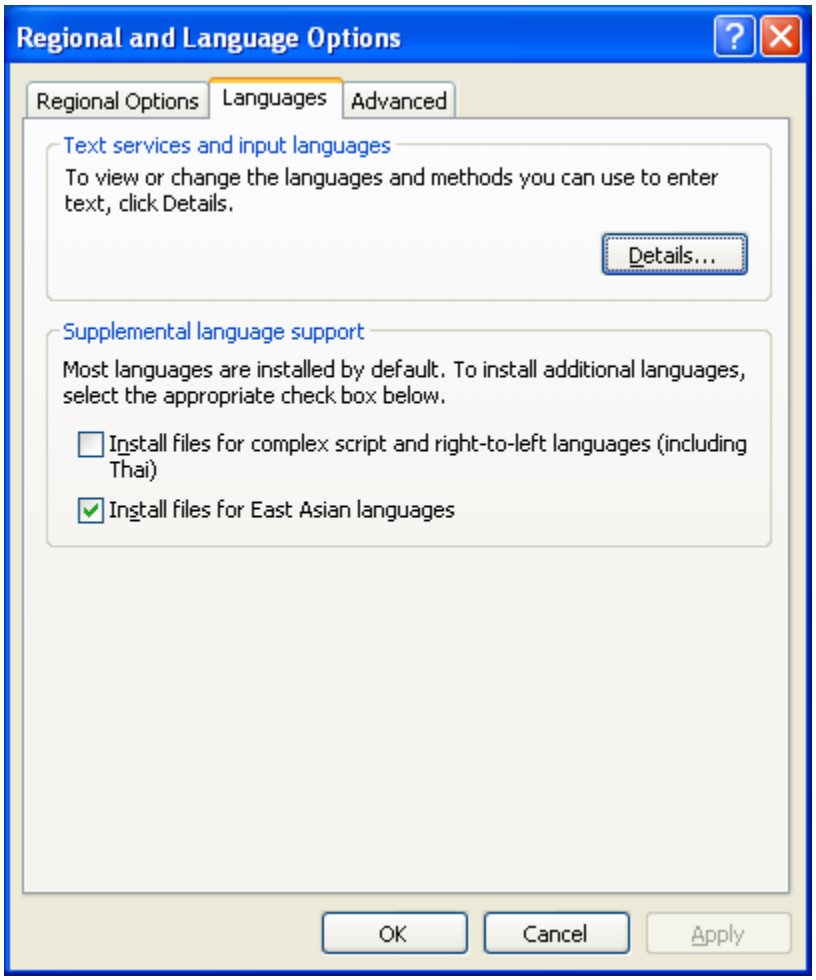
14 Appendix C: Windows IME Setup

This appendix provides instructions for installing input method support on your Windows system.

14.1 Windows XP

We demonstrate how to install Japanese IME on a Windows XP machine. The process of installing other IMEs (Chinese Simplified, Chinese Traditional, Korean) is similar.

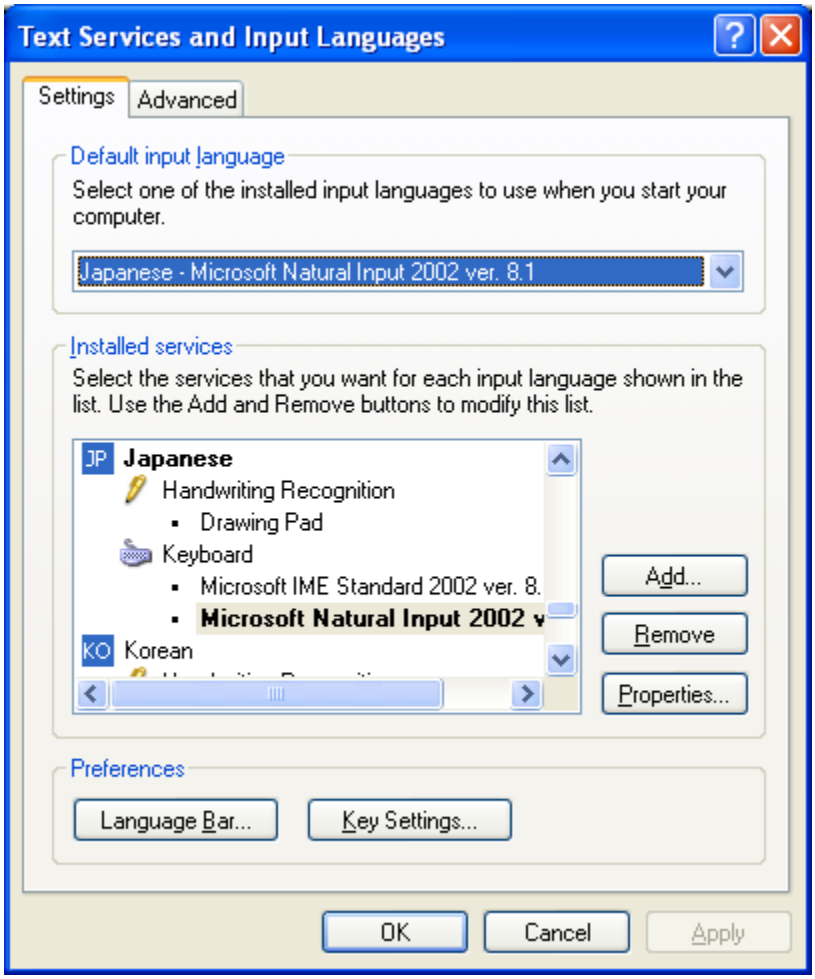
To install Japanese, start by opening the **Control Panel** (Click the "Start Menu" and chose the "Control Panel"). Then click the "**Regional and Language Options**" icon. You should then see the "**Regional and Language Options**" dialog box as shown below.



Two things need to be done here:

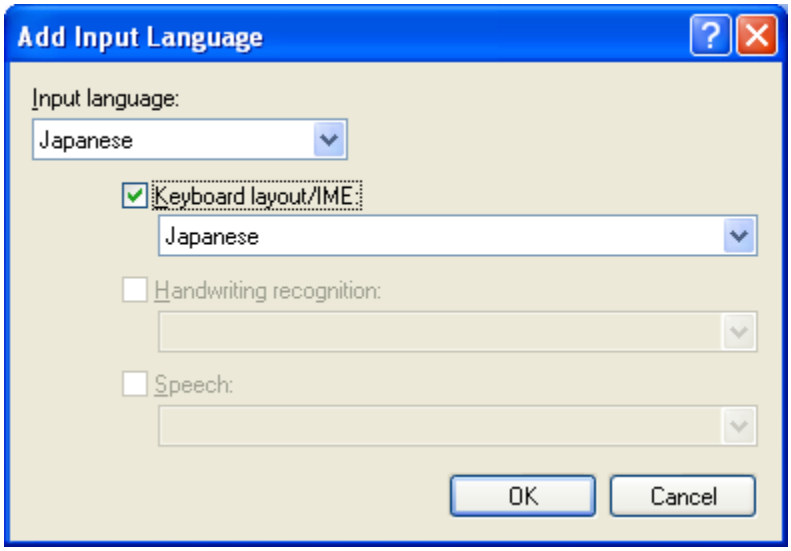
1. Click on the "**Install files for East Asian Languages**" check box
   (This ensures that the correct fonts are installed).

2. Then click the "**Details**" button.
   You should then see the "**Text Services and Input Languages**" dialog box.



Click the "**Add...**" button.
You should now see the "**Add Input Language**" dialog box, show on the next page.

Chose "**Japanese**" in the "**Input language**" combo box as illustrated below.



Still in the "**Add Input Language**" dialog box, in the "**Keyboard layout/IME**" combo box and chose "**Microsoft IME Standard 2002 ver. 8.1**".

Finally click the "**OK**" button.

You may be asked to put your Windows XP CD-ROM in the CD drive so that the relevant files can be copied across to your hard drive. **NOTE:** You **cannot** install the IME without the CD.

After rebooting you should see Japanese icon in the language bar. Select Hiragana in the input mode tab.

14.2 Windows Vista

The process of installing IMEs on Windows Vista is similar to that on XP. One noticeable difference is that the "**Text Services and Input Languages**" dialog box is now summoned by using the "**Change keyboards…**" button of the **"Keyboards and Languages**" tab of regional options. The rest of the input language configuration should work identically to XP. With Vista, the files needed for installation are already present on the system, so they just need to be configured for use.

15 Appendix D: IME Messages on Windows

As was discussed in section on IME code integration, end user is responsible for wrapping all of the WM_IME messages into GFx::IMEWin32Event and passing them to the movie view. This section presents a brief introduction to these messages and the role they play in GFxIME.

- **WM_IME_SETCONTEXT**: Sent by IME to an application whenever the state of the IME windows changes (resizing/moving etc). According to the MSDN documentation, one of the parameters to this message contains certain flags that can be set/unset to show/hide IME windows. We found that in practice this doesn't work.

- **WM_IME_STARTCOMPOSITION**: Sent immediately before the IME generates the composition string as a result of a keystroke.

- **WM_IME_ENDCOMPOSITION**: Sent to an application when the IME ends string composition. Note that with some Chinese IMEs the WM_END_COMPOSITION message is not sent even when the composition string is finalized.

- **WM_IME_COMPOSITION**: The **WM_IME_COMPOSITION** message is sent to an application when the IME changes composition status as a result of a keystroke. This message can be processed to obtain information about various types of composition events - change in the result string, clause information, etc. This is documented in the MSDN documentation. Note that on Windows XP, for certain IMEs (Chinese (Traditional) - New Phonetic and Chinese (Traditional) - New ChangJie for example), a space is inserted at the caret position in the text field as the reading window text is being input. For other IME's (Chinese (Simplified) – QuanPin and Chinese (Traditional) – DaYi), the composition text is empty until the user is finished with typing in text for the reading window and sends the reading text to the IME for conversion into a composition character (usually by pushing the space key). On Windows Vista, the behavior is more consistent.

- **WM_IME_NOTIFY**: The **WM_IME_NOTIFY** message is sent to an application to notify it of changes to the IME window. The wParam parameter specifies the type of change. This message tells us about events like opening/closing the candidate window, repositioning the composition/input/status windows. Note that on Windows XP, some IME's use wParam = IMN_PRIVATE for handling backspace and space characters which is undocumented behavior.

Detailed information about these messages and other IME related information can be found in the MSDN docs on Microsoft's website.